



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Проф. др Игор Дејановић

Софтверски обрасци и компоненте

скрипта за предмет

Нови Сад, 2025

Садржај

1	Увод	1
1.1	Увод	1
1.2	Софтверски обрасци	2
1.3	Софтверски антиобрасци	4
2	Креациони обрасци	7
2.1	Фундаментални обрасци	7
2.2	Delegation	7
2.3	Interface	10
2.4	Interface and Abstract Class	11
2.5	Креациони обрасци	13
2.6	Factory Method	13
2.7	Abstract Factory	14
2.8	Prototype	15
2.9	Builder	16
2.10	Singleton	19
2.11	Литература	20
3	Структурални обрасци	21
3.1	Структурални софтверски обрасци	21
3.2	Adapter	21
3.3	Bridge	23
3.4	Composite	25
3.5	Decorator	27
3.6	Proху	29
3.7	Литература	31
4	Обрасци понашања	33
4.1	Обрасци понашања	33
4.2	Iterator	33
4.3	Command	34
4.4	Mediator	36
4.5	Memento	37
4.6	Observer	39
4.7	State	41
4.8	Strategy	42
4.9	Template Method	43
4.10	Visitor	45

4.11	Chain of Responsibility	47
4.12	Литература	48
5	Микросервисни обрасци	49
5.1	Увод	49
5.2	Обрасци за рад са базама података	51
5.3	Обезбеђивање конзистенције	54
5.4	Постављање упита	60
5.5	Комуникација	62
5.6	Откривање сервиса	67
5.7	Литература	68
6	Развој базиран на компонентама	71
6.1	Razvoj softvera	71
6.2	Modularizacija	71
6.3	Softverske komponente	71
6.4	Razvoj baziran na komponentama	71
6.5	Šta je problem sa softverom?	71
6.6	Integracija kao preduslov ponovne iskoristljivosti komponenti	72
6.7	Preduslovi uspešne komponentizacije	72
6.8	Vrste softvera prema načinu dobavljanja	72
6.9	Namenski izrađen softver	72
6.10	Standardni softver	72
6.11	Problem sa standardnim softverom	73
6.12	Komponentni softver -- hibridni pristup	73
6.13	Evolutivni razvoj	73
6.14	Univerzalne komponente?	73
6.15	Apstraktna arhitektura softvera baziranog na komponentama	74
6.16	Platforma	74
6.17	Servisi	74
6.18	Konzumenti i pružaoci servisa	74
6.19	Literatura	74
7	Креирање компоненти у Пajтону	75
7.1	Мотивација	75
7.2	Конзолне скрипте	75
7.3	Прикључне тачке (<i>Entry Points</i>)	76
7.4	Литература	79

Глава 1

Увод

Садржај

1.1 Увод

1.1.1 Развој софтвера

- Развој софтвера је тежак а развој софтвера на такав начин да се решење може поново употребити је још теже.
- Дobar, проширив и флексибилан дизајн је тешко а вероватно и немогуће одрадiti из *прве* за неискусног пројектанта.

1.1.2 Искуство

- Искусни пројектанти, тј. експерти, ипак праве добар дизајн.
- Експерти знају нешто што почетници не знају. Шта је то?

1.1.3 Поновно коришћење добрих решења

Експерти ће поново примењивати решења која су се показала као добра у прошлости.

1.1.4 Поновна искористљивост - *Reusability*

Дефиниција са *The Free Dictionary*:

The ability to use all or the greater part of the same programming code or system design in another application.

- <http://encyclopedia2.thefreedictionary.com/reusability/>

1.1.5 Поновна искористљивост - Зашто?

- Да не би измишљали *поп*у воду.
- Употреба већ постојећих решења - бржа изградња софтвера.
- Већ коришћен софтвер је боље тестиран - робусност.

1.1.6 Поновна искористљивост - Како?

- На нивоу кода - библиотеке, оквири за развој (*frameworks*), софтверске компоненте итд.
- На нивоу дизајна - дизајн обрасци.

1.2 Софтверски обрасци

1.2.1 Шта је образац?

Christopher Alexander и сар. су написали*:

Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.

* Christopher Alexander, Sara Ishikawa, Murray Silverstein, Max Jacobson, Ingrid Fiksdahl-King, and Shlomo Angel. *A Pattern Language*. Oxford University Press, New York, 1977.

1.2.2 ...ili у слободном преводу

Сваки образац описује проблем који се појављује наново у нашем окружењу, и затим описује суштинску решења датог проблема на такав начин да решење можете применити милион пута а да никада не решите проблем на потпуно идентичан начин.

1.2.3 Шта је софтверски образац?

Рецепт настао на бази кумулираног експертског знања и искуства у решавању одређеног рекурентног проблема у развоју софтвера који описује проблем, решење и контекст у коме је решење примењиво као и предности и мане решења.

1.2.4 Градивни елементи софтверског обрасца

- *Назив* - Обезбеђује лако референцирање на образац. Чини део вокабулара и омогућава комуникацију.
- *Проблем* - Опис проблема и контекста у коме се образац може применити. Често се наводи и листа предуслова за употребу обрасца.
- *Решење* - Елементи, међусобне везе, сарадња и одговорност. Апстрактно је да би могло бити примењено у различитим конкретним контекстима.
- *Последице* - описују шта је то што добијамо а шта губимо употребом датог обрасца (*trade-offs*).

1.2.5 У чему је предност употребе софтверских обрасца?

- Учење на туђим грешкама.
- Мање искусни пројектанти могу правити добар дизајн.
- Лакша комуникација. Конзистентан вокабулар.
- Лакша анализа дизајна сложеног софтвера. Обрасци имају своју **мустру**.

- Лакше укључивање нових људи на пројекат.
- Познавање унапред свих евентуалних проблема које решење може да изазове.

1.2.6 Избор програмског језика

Избор програмског језика и програмске парадигме са становишта софтверских образаца је важан!

1.2.7 Организација

- Класификација и каталогизација у циљу лакшег проналажења.
- Различити аутори - различите класификације.
- Различити домени - различити обрасци.

1.2.8 Класификација ОО дизајн образаца

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Flyweight (195) Observer State Strategy Visitor

- E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994

1.2.9 Класификација ОО дизајн образаца

Класификација ОО дизајн образаца према [1]:

- Фундаментални

- Креациони
- Партициони
- Структурални
- Обрасци понашања
- Обрасци за конкурентни дизајн

1. M. Grand, *Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML*, John Wiley & Sons, Inc., vol. 1, 2002

1.3 Софтверски антиобрасци

1.3.1 Софтверски антиобрасци

- У развоју софтвера антиобрасци представљају обрасце који су се показали неефикасним и контрапродуктивним у пракси *.
- Још се зову и *pitfalls* или *dark patterns*.
- * [Anti-pattern](http://en.wikipedia.org/wiki/Anti-pattern)¹, From Wikipedia, the free encyclopedia

1.3.2 Како их препознати?

Антиобрасци исказују следеће особине:

- Рекурентан образац акције, процеса или структуре који је иницијално изгледао као добар али је у пракси показао да доноси више проблема него решења,
- Рефакторисано решење проблема постоји, јасно је документовано и доказано у пракси.

1.3.3 Неки од примера антиобразаца

Примери антиобразаца су*:

- *Cargo cult programming* - Коришћење образаца и метода без разумевања због чега се користе.
- *God object* - Концентрисање превише функционалности у један елемент дизајна (најчешће класу).
- *Big ball of mud* - Систем без јасне структуре.
- *Copy and paste programming* - Копирање и измена постојећег кода уместо креирања генеричког решења.

¹<http://en.wikipedia.org/wiki/Anti-pattern>

- *Not Invented Here (NIH) sindrom* - Тенденција ка измишљању **топле воде** уместо коришћења постојећег, опробаног решења.
- *Premature optimization* - Оптимизовање софтвера прерано на уштрб доброг дизајна и функционалности.
- *Dependency hell* - Проблеми са верзијама зависних компоненти и софтвера.
- * За шири списак видети <http://en.wikipedia.org/wiki/Anti-pattern>

1.3.4 Литература

- E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994
- M. Grand, *Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML*, John Wiley & Sons, Inc., vol. 1, 2002
- [Anti-pattern](http://en.wikipedia.org/wiki/Anti-pattern)², From Wikipedia, the free encyclopedia

²<http://en.wikipedia.org/wiki/Anti-pattern>

Глава 2

Креациони обрасци

Садржај

2.1 Фундаментални обрасци

2.1.1 Фундаментални обрасци

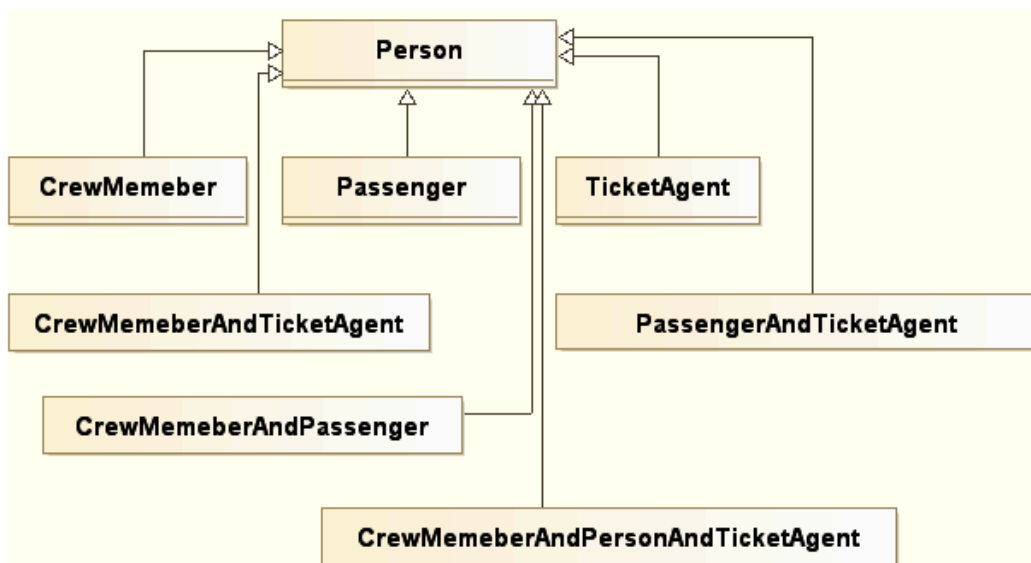
- Обрасци који се користе за изградњу сложенијих образаца.
- Често подржани програмским језиком.

2.2 Delegation

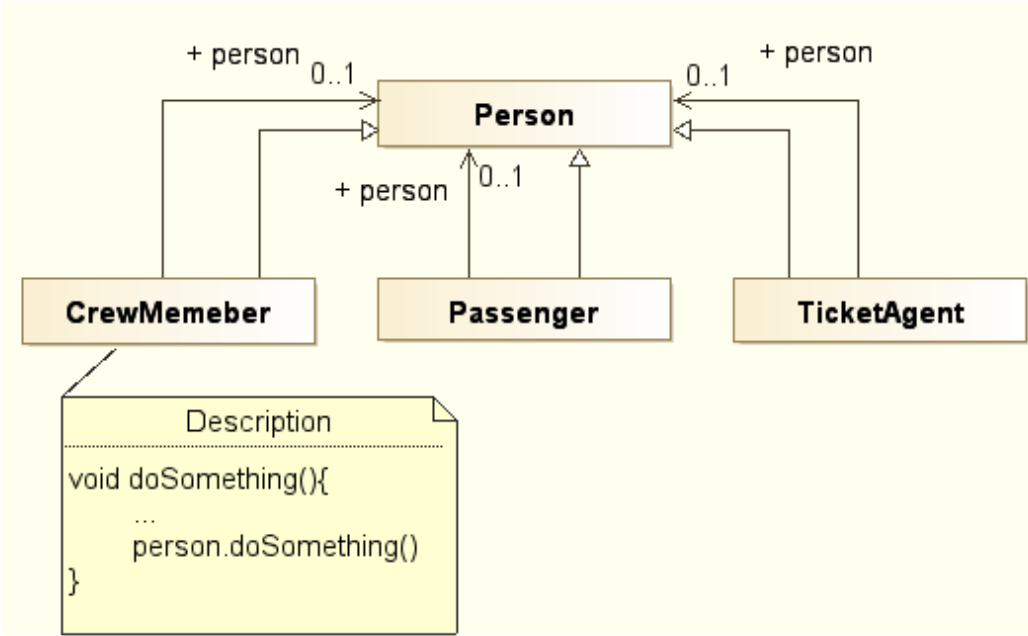
2.2.1 Delegation

- Општи начин проширења функционалности класе или објекта.
- Користи се у ситуацијама када наслеђивање није довољно флексибилно, посебно у ситуацији када програмски језик омогућава само једноструко наслеђивање.
- Динамичко проширење при инстанцирању класе дефинисањем делегата.

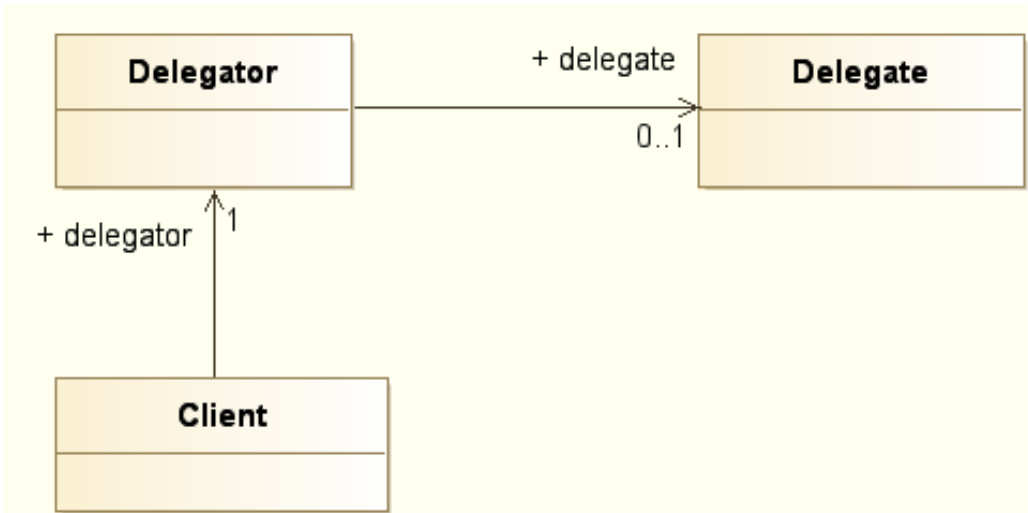
2.2.2 Пример проблема



2.2.3 Пример решења



2.2.4 Структура обрасца



2.2.5 Пример

```

interface I {
    void f();
    void g();
}

class A implements I {
    public void f() { System.out.println("A: doing f()"); }
    public void g() { System.out.println("A: doing g()"); }
}

class B implements I {
    public void f() { System.out.println("B: doing f()"); }
    public void g() { System.out.println("B: doing g()"); }
}

class C implements I {
    // delegation
    I i = new A();

    public void f() { i.f(); }
    public void g() { i.g(); }
}

// normal attributes
public void toA() { i = new A(); }
public void toB() { i = new B(); }

public class Main {
    public static void main(String[] args) {
        C c = new C();
        c.f();
        // output: A: doing f()
        c.g();
        // output: A: doing g()
        c.toB();
        c.f();
        // output: B: doing f()
        c.g();
        // output: B: doing g()
    }
}

```

- Извор: Wikipedia

2.2.6 Када користити?

- Наслеђивање је статичка веза. Потребно је да објекат *мења класу* у време извршавања.

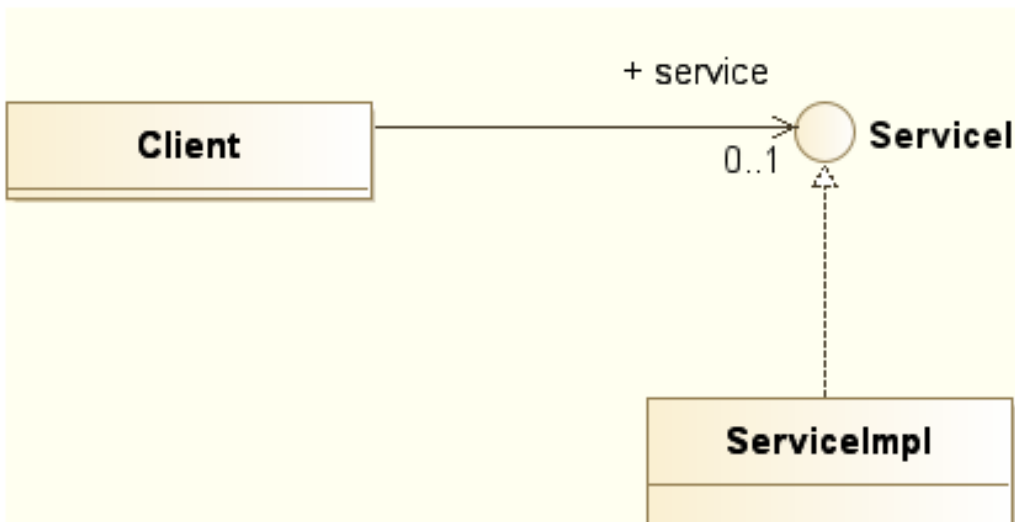
- Потребно је симулирати вишеструко наслеђивање. Делегатор може имати више делегата.
- Желимо да наследимо одређену класу али она није под нашом контролом. Делегација је у том случају безбеднија јер нас штити од некомпатибилних промена надкласе.

2.3 Interface

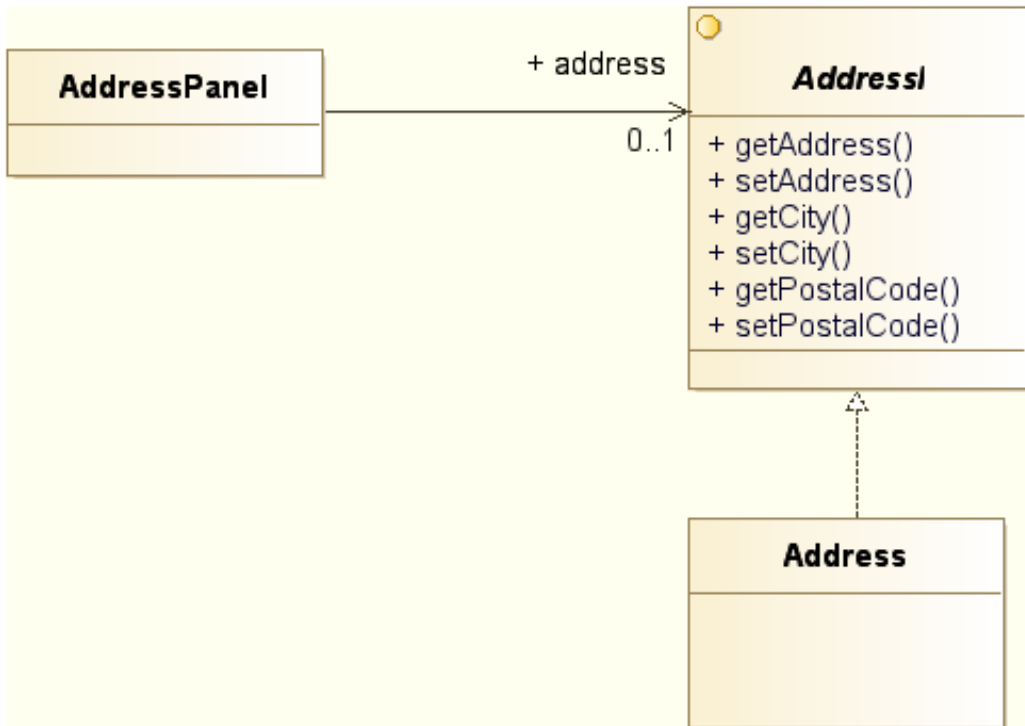
2.3.1 Interface

- Желимо да учинимо клијента независним од класа које обезбеђују сервис тако да можемо изменити класе без утицаја на клијента.
- Сервисне класе већ наслеђују неке класе а вишеструко наслеђивање није подржано.

2.3.2 Структура обрасца



2.3.3 Пример



2.3.4 Шта добијамо?

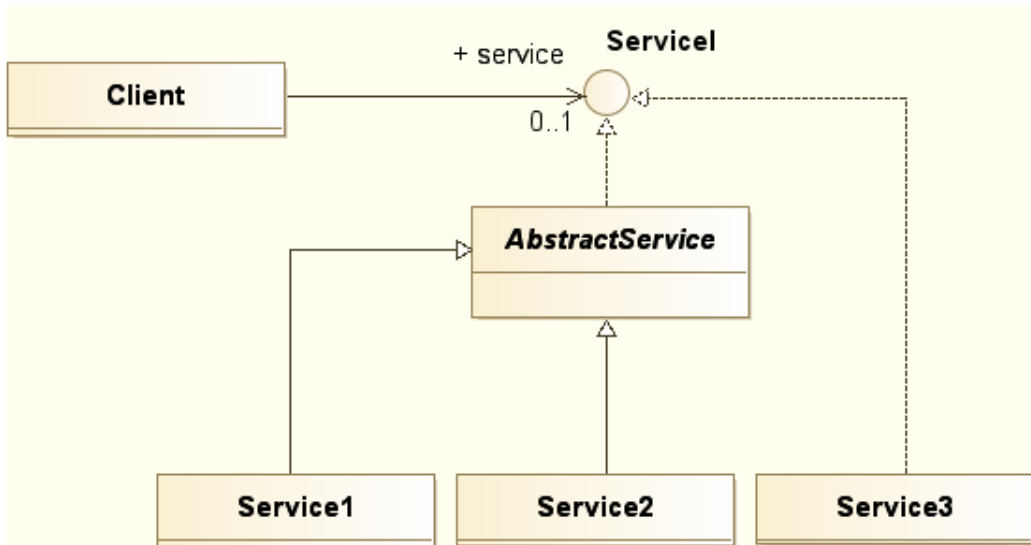
- Сервисна класа може бити замењена без утицаја на клијента.
- Сервисна класа може имплементирати више интерфејса и наслеђивати произвољну класу.

2.4 Interface and Abstract Class

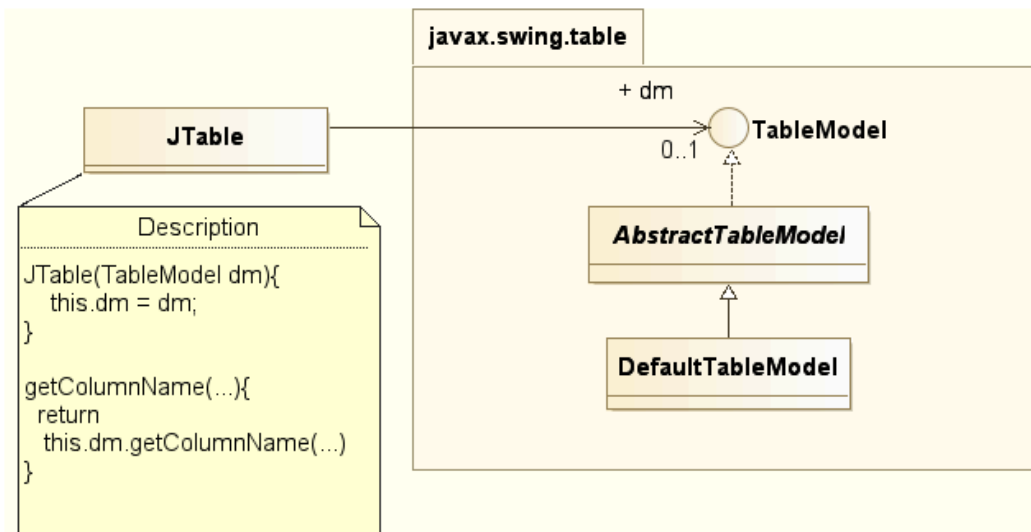
2.4.1 Interface and Abstract Class

- Проширење *Interface* обрасца.
- Желимо да клијента учинимо независним од хијерархије класа које имплементирају сервис на такав начин да можемо мењати класе које имплементирају сервис без утицаја на клијента.
- Истовремено желимо да дефинишемо апстрактну имплементацију сервиса тако да конкретни сервиси могу да редефинишу/допуне базну имплементацију.

2.4.2 Структура обрасца



2.4.3 Пример



2.4.4 Шта добијамо?

- Конкретан сервис може бити базиран на апстрактној имплементацији али и не мора. Довољно је да имплементира интерфејс сервиса.
- Сервисна класа може имплементирати више интерфејса и може наслеђивати произвољну класу.

- Апстрактна класа обезбеђује базичну имплементацију чиме се развој нових сервисних класа убрзава.

2.5 Креациони обрасци

2.5.1 Креациони обрасци

- Апстракција и локализација процеса инстанцирања објеката.
- Чине систем независним од начина креирања и композиције објеката.

2.5.2 Базичне особине

Базичне особине креационих образаца су следеће:

- Енкапсулирају знање о конкретним класама које систем користи.
- Скривају начин на који се врши инстацирање класа и композиција објеката.

2.5.3 Каталог креационих образаца

Према [1] креациони обрасци су следећи:

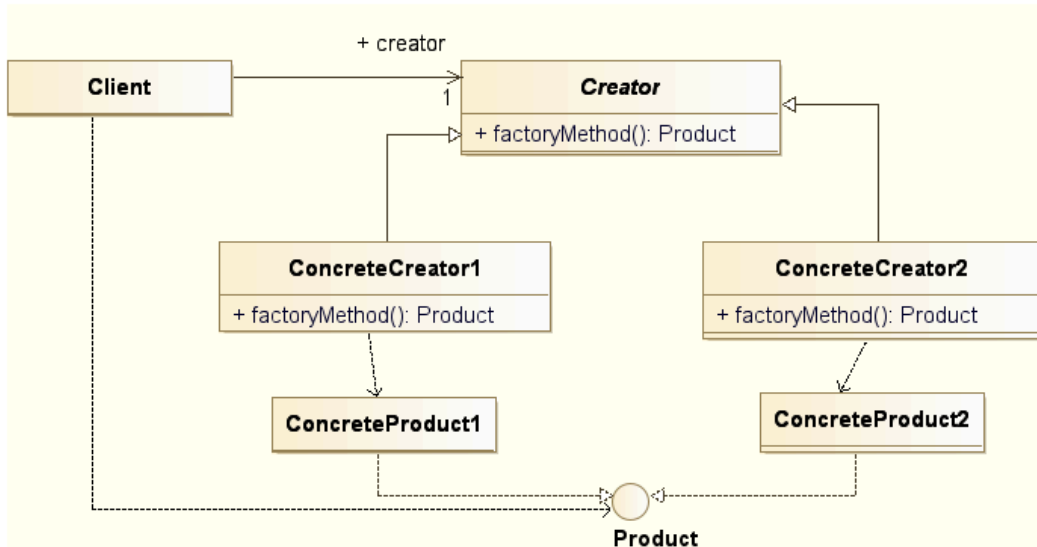
- Factory Method
- Abstract Factory
- Prototype
- Builder
- Singleton
- E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994

2.6 Factory Method

2.6.1 Factory Method

- Желимо да одвојимо клијента од конкретне имплементације путем Интерфејса.
- Желимо да клијент креира објекте на такав начин да “не зна” коју конкретну класу инстанцира.

2.6.2 Структура



2.6.3 Шта добијамо?

- Инстанцирање конкретних објеката је локализовано. Програмски код није јако спрегнут са типом конкретних инстанци.
- Класа може делегирати креирање конкретних објеката које користи на своје подкласе чиме се постиже већа флексибилност.

2.6.4 А шта губимо?

- Релативно велики број класа.
- Сваки нови Производ захтева новог Креатора.

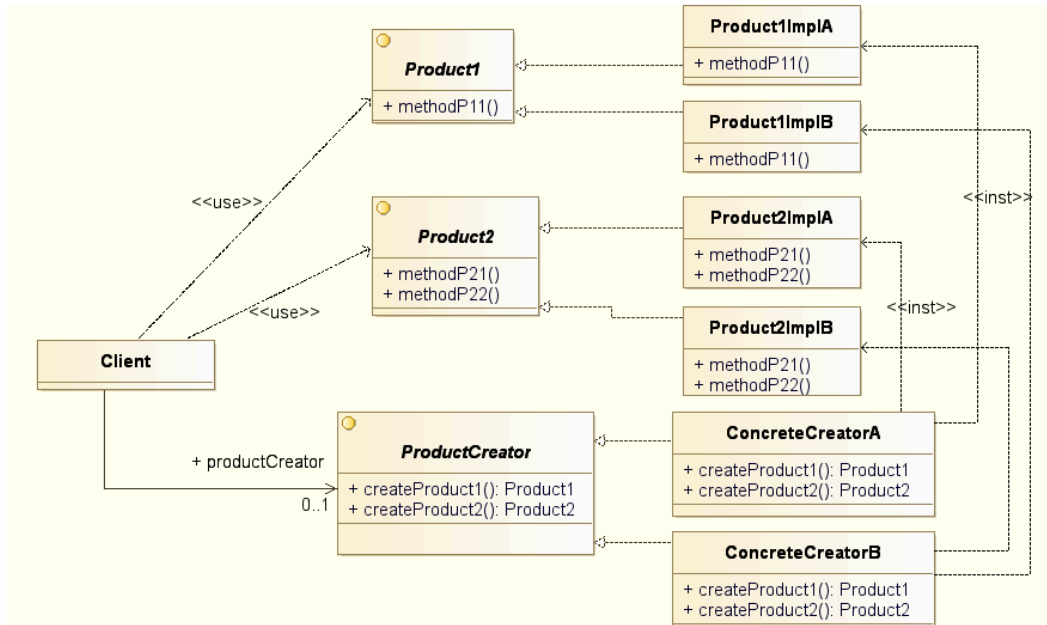
2.6.5 Демонстрација обрасца *Factory Method* на примеру у Јави

2.7 Abstract Factory

2.7.1 Abstract Factory

- Проширење *Factory Method* обрасца.
- Креирање фамилије објеката сличних по функцији коју врше у апликацији при чему клијент (објекат који их користи) “не зна” конкретне класе објеката.

2.7.2 Структура



2.7.3 Шта добијамо?

- Инстанцирање фамилије конкретних објеката је локализовано.
- Програмски код није јако спрегнут са типом конкретних инстанци.
- Промена фамилије конкретних инстанци могућа је изменом програмског кода на месту где се инстанцира конкретна фабрика објеката (најчешће једна линија кода).
- У одређеним случајевима могућа је динамичка промена фабрике објеката (нпр. [Swing PLAF³](https://en.wikipedia.org/wiki/Pluggable_look_and_feel)).

2.7.4 А шта губимо?

- Релативно велики број класа.
- Сваки нови Производ захтева нову методу у свим Креаторима и свака нова фамилија производа захтева новог конкретног Креатора и нове конкретне производе.

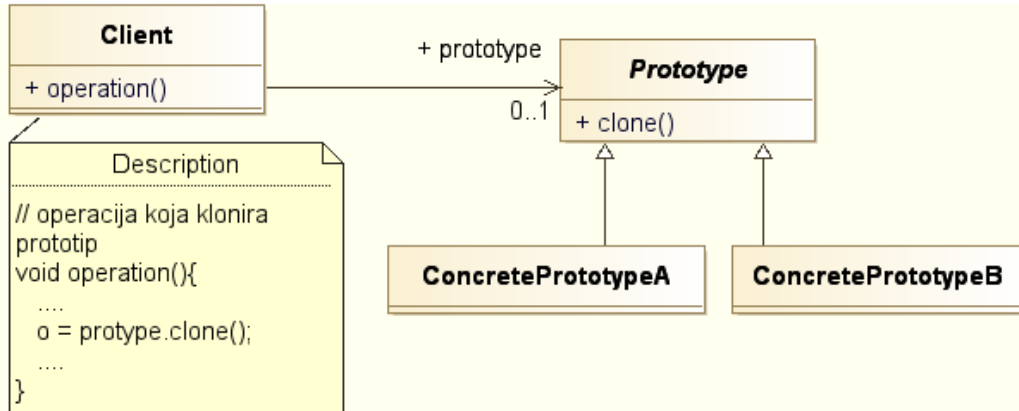
2.8 Prototype

2.8.1 Prototype

Креирање објекта копирањем већ постојеће инстанце.

³https://en.wikipedia.org/wiki/Pluggable_look_and_feel

2.8.2 Структура



2.8.3 Шта добијамо?

- Елиминисемо потребу за постојањем креатор класа и њених наследница при коришћењу *Factory Method* дизајн шаблона - смањује се број потребних класа.
- Ако је инстанцирање објеката превише “скуп” са становишта потрошње процесорског времена и меморије, клонирање већ креираних објеката може довести до убрзања и смањења потрошње меморије.
- Уколико нам требају инстанце објеката чије стање се разликује од иницијалног али је слично већ постојећим једноставније је клонирање постојећег, сличног, објекта и измена потребних атрибута од креирања и пуне иницијализације новог објекта.

2.8.4 На шта треба обратити пажњу?

- Клонирање се најчешће реализује тзв. “дубоким копирањем” (енг. *deep copy*).
- Јавина clone операција је реализована као “плитко копирање” (енг. *shallow copy*).

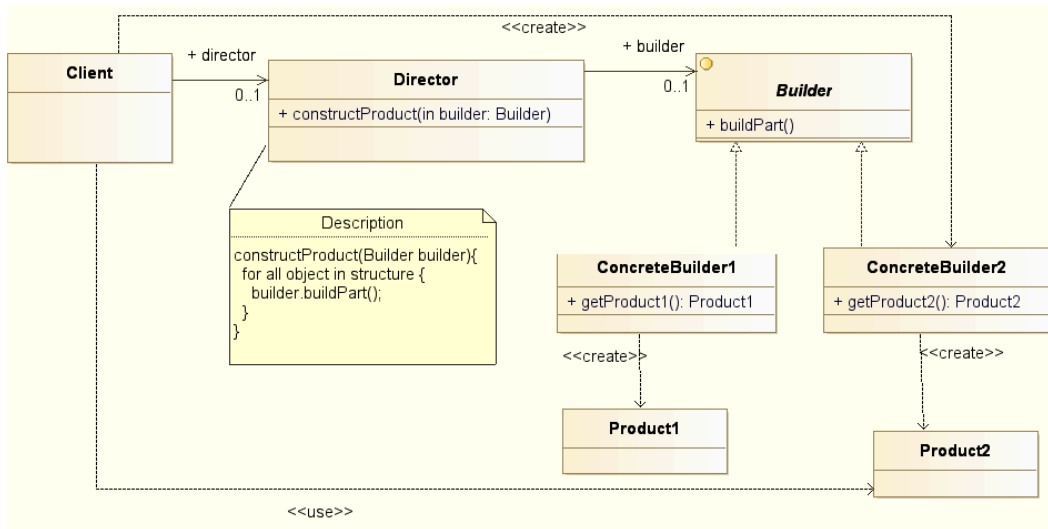
2.8.5 Демонстрација обрасца *Prototype* на примеру у Јави

2.9 Builder

2.9.1 Builder

Конструкција сложених објеката поделом надлежности на онога ко “зна” како треба направити структуру сложеног објекта и онога ко “зна” како треба креирати појединачне делове.

2.9.2 Структура



2.9.3 Шта добијамо?

Раздвајамо конструкцију сложеног објекта (најчешће *Composite*) од његове репрезентације тако да исти поступак изградње може резултовати различитим репрезентацијама.

2.9.4 Када користити?

- Ако је потребно учинити алгоритам за креирање сложених објеката независним од конкретних делова који сачињавају објекат.
- Ако процес конструкције мора обезбедити различите репрезентације објекта који се изграђује.

2.9.5 Напомена

Постоји и друга врста *Builder* обрасца где се објекат гради у корацима употребом помоћног *Builder* објекта.

Пример у Јави:

```
public class Car {
    private String make;
    private String model;
    private int year;
    private String color;
    private int horsepower;

    private Car(Builder builder) {
        this.make = builder.make;
        this.model = builder.model;
        this.year = builder.year;
        this.color = builder.color;
        this.horsepower = builder.horsepower;
    }

    public static class Builder {
        private String make;
        private String model;
        private int year;
        private String color;
        private int horsepower;

        public Builder withMake(String make) {
            this.make = make;
            return this;
        }

        public Builder withModel(String model) {
            this.model = model;
            return this;
        }
        ...

        public Car build() {
            return new Car(this);
        }
    }
    ...
}
```

```

@Override
public String toString() {
    return "Car{" +
        "make='" + make + '\'' +
        ", model='" + model + '\'' +
        ", year=" + year +
        ", color='" + color + '\'' +
        ", horsepower=" + horsepower +
        '}';
}

public static void main(String[] args) {
    Car car = new Car.Builder()
        .withMake("Toyota")
        .withModel("Camry")
        .withYear(2020)
        .withColor("Blue")
        .withHorsepower(200)
        .build();

    System.out.println(car);
}

```

Car{make='Toyota', model='Camry', year=2020, color='Blue', horsepower=200}

Може се комбиновати и са [Typestate обрасцем](#)⁴ у циљу боље контроле могућих конфигурација.

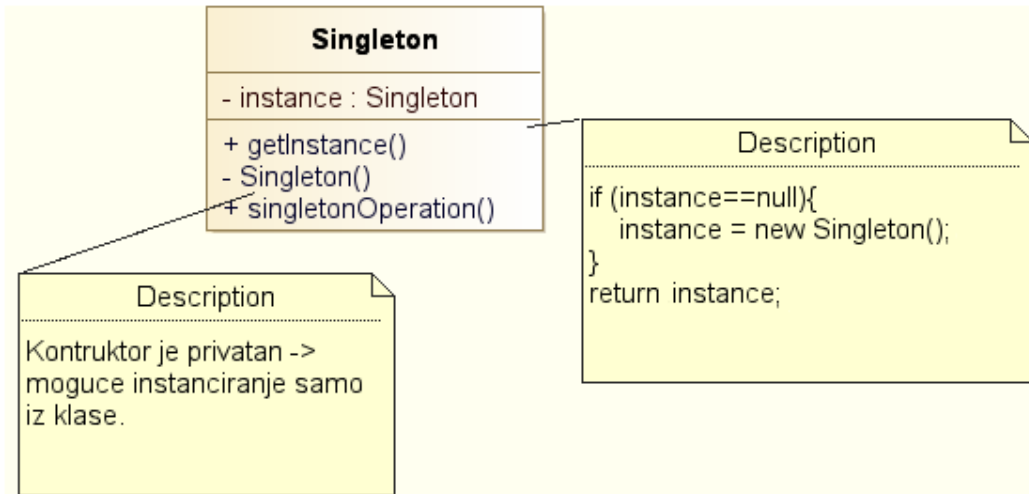
2.10 Singleton

2.10.1 Singleton

- Потребно је да постоји само једна инстанца класе у систему. На пример: logger, регистар, јединствена конекција ка бази и сл.
- Јединична инстанца мора бити доступна свим клијентима.

⁴<https://docs.rust-embedded.org/book/static-guarantees/typestate-programming.html>

2.10.2 Структура



2.10.3 Шта добијamo?

- Контролисани приступ јединственој инстанци.
- Наслеђивање класе јединствене инстанце и омогућавање реконфигурације система и у време извршавања (*run-time*).
- Ако је накнадно потребно, може се обезбедити постојање више од једне инстанце. На пример, *Connection Pool*.

2.10.4 Напомена

- *Singleton* се данас сматра анти-обрасцем.
- Два главна разлога:
 - *Singleton* је глобални објекат – глобалне варијабле лоше утичу на квалитет дизајна и на конкурентност у контексту вишенитног програмирања.
 - *Singleton* се тешко *mock-up*-ује (клијентски код је чврсто спрегнут са *Singleton* објектом) па је тестирање кода који га користи веома тешко. Овај проблем се може решити коришћењем *Dependency Injection* обрасца.

2.11 Литература

- E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994
- M. Grand, *Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML*, John Wiley & Sons, Inc., vol. 1, 2002

Глава 3

Структурални обрасци

Садржај

3.1 Структурални софтверски обрасци

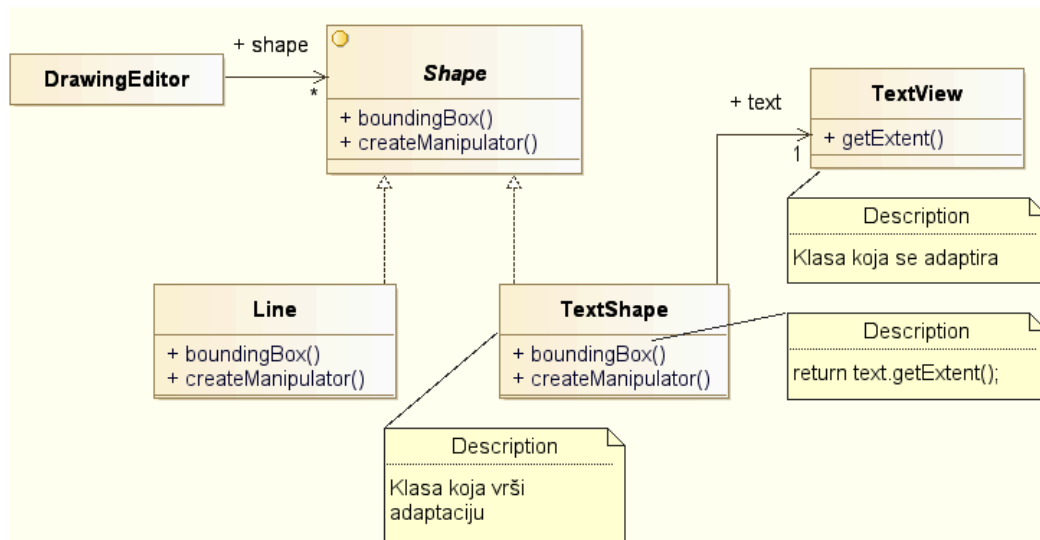
Баве се удруживањем објеката и класа, коришћењем наслеђивања, композиције и делегације, у циљу формирања сложенијих структура.

3.2 Adapter

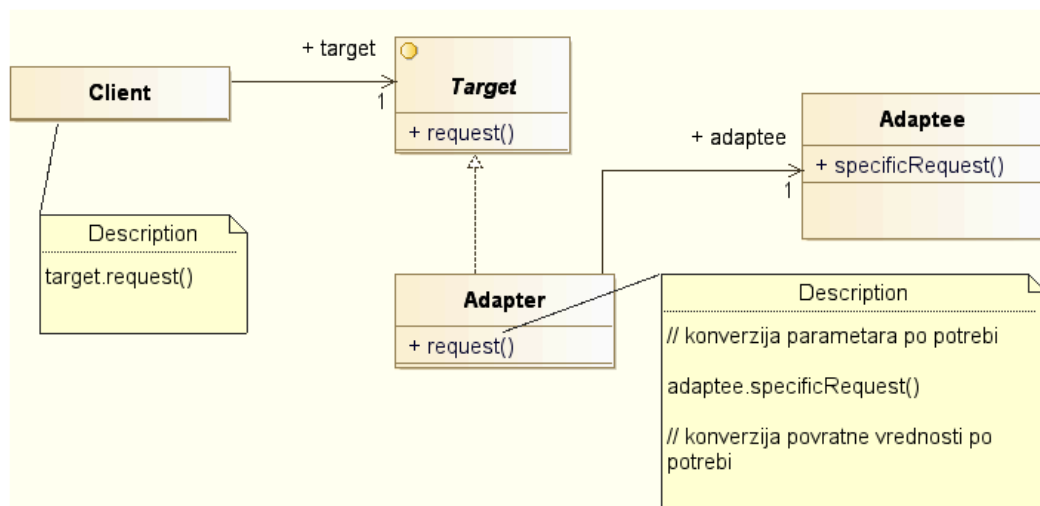
3.2.1 Adapter

- Прилагођавамо компоненту чија нам је функционалност потребна али њен интерфејс не одговара нашим захтевима.
- Нпр. прилагођавање COTS (*Commercial Of-The-Shelf*) компоненти нашем систему или прилагођавање старих компоненти захтевима новог система.
- Креирање компоненте тако да је могуће користити у различитим, унапред непознатим, сценаријима.
- Познат још и као *Wrapper*.

3.2.2 Пример



3.2.3 Структура



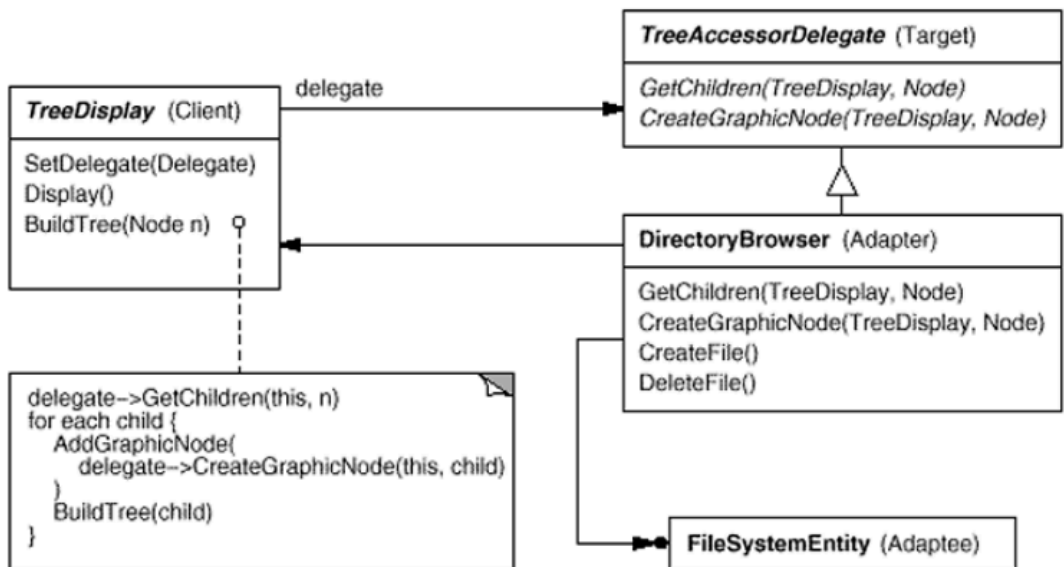
3.2.4 Две верзије

- *Базиран на класама и наслеђивању* – Класа која се адаптира се наслеђује и класа наследница имплементира одговарајући интерфејс.
- *Базиран на објектима и делегацији* – Објекту који се адаптира се делегирају позиви од стране адаптер објекта.

3.2.5 Напомене

- Количина посла коју адаптер ради варира од обичне конверзије параметара и назива метода до подршке за потпуно другачији сет операција.
- Измењиви адаптери (*Pluggable adapters*) – Креирање компоненте тако да се унапред предвиди адаптација објеката са којима треба да сарађује иако унапред објекти и њихове класе нису познати. Пример *TreeDisplay*.¹
- E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994

3.2.6 Пример - TreeDisplay



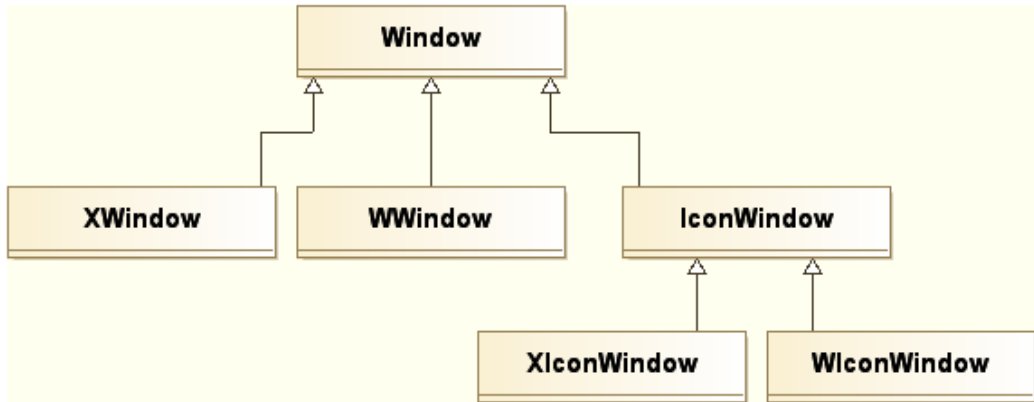
- E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994

3.3 Bridge

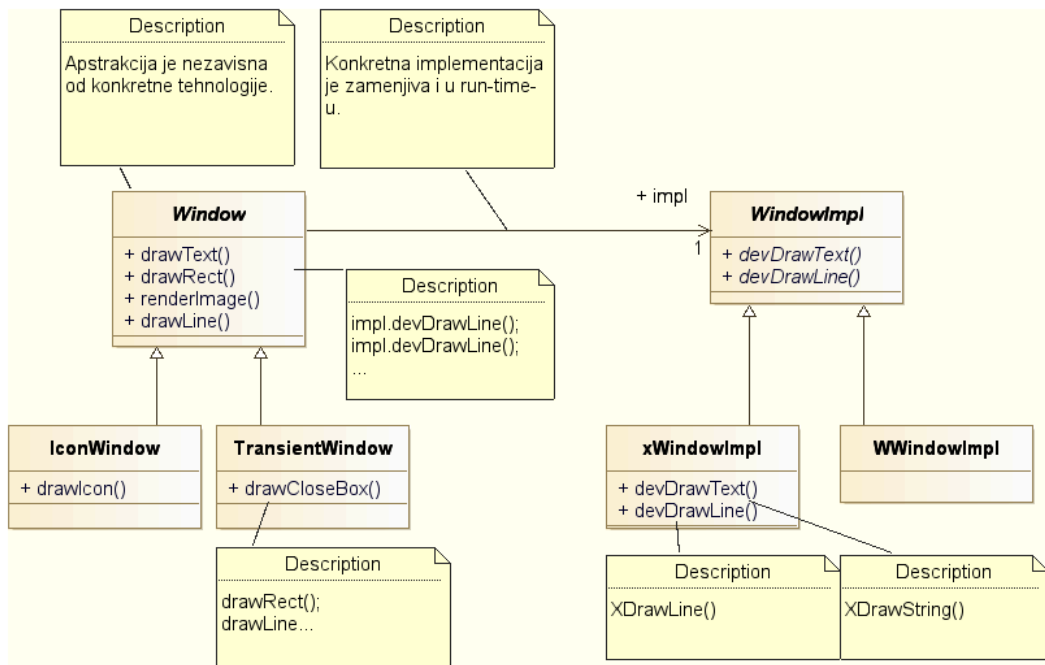
3.3.1 Bridge

Раздвајање абстракције од имплементације тако да се могу независно мењати.

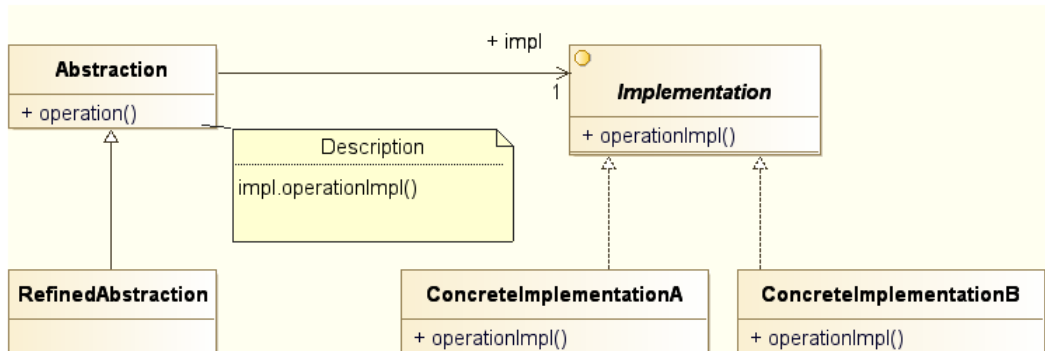
3.3.2 Пример проблемы



3.3.3 Примена *Bridge* обрасца



3.3.4 Структура



3.3.5 Када кориситити

- Када желимо избећи трајно везивање апстракције и имплементације. На пример, ако је потребна измена имплементације у време извршавања апликације.
- Када желимо да омогућимо независно проширење апстракције и имплементације кроз наслеђивање.
- Када имамо сложене хијерархије класа као што је показано у примеру.

3.3.6 Инстанцирање конкретне имплементације

Одлука о инстанцирању конкретне имплементације:

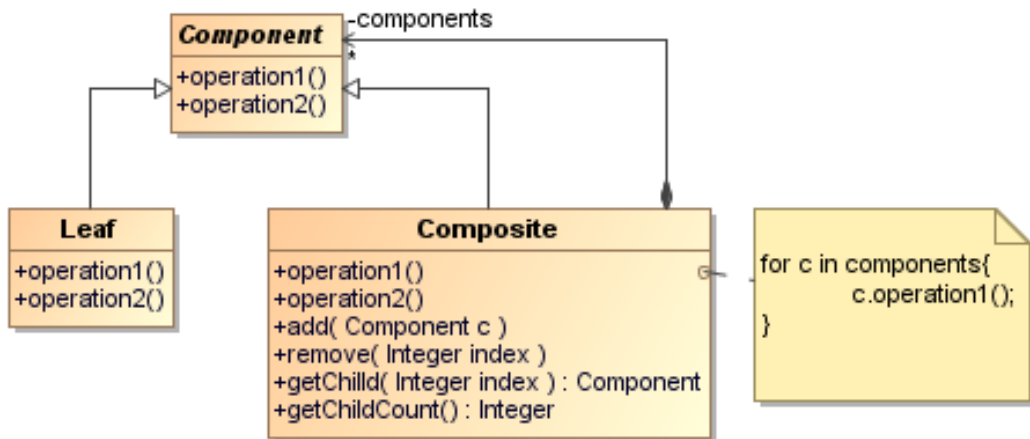
- Конструктор апстракције путем параметра (ако зна за све имплементације).
- Делегирање надлежности другом објекту – видети *Factory Method* - уколико је потребно учинити апстракције несвесне конкретних имплементација.

3.4 Composite

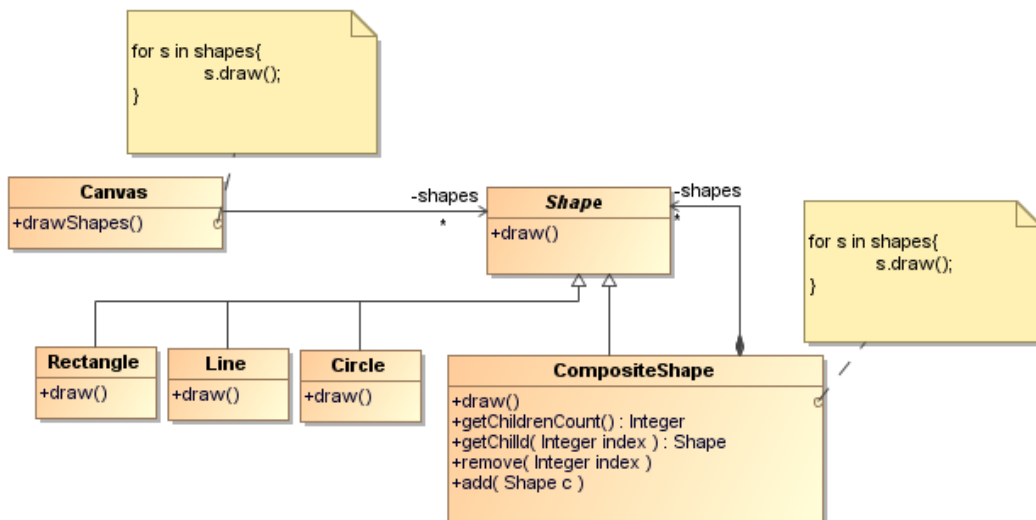
3.4.1 Composite

- Композиција објеката у структуре облика стабла.
- Униформно третирање простих и сложених објеката.

3.4.2 Структура обрасца



3.4.3 Пример



3.4.4 Напомена

GOF књига – методе за манипулацију *child* елементима у **Component** апстрактној класи.

3.4.5 Када користити?

- Ако желите да креирате хијерархије објеката типа целина-део.
- Желите да клијент може да игнорише разлике између композитних и простих објеката и третира их на униформан начин.

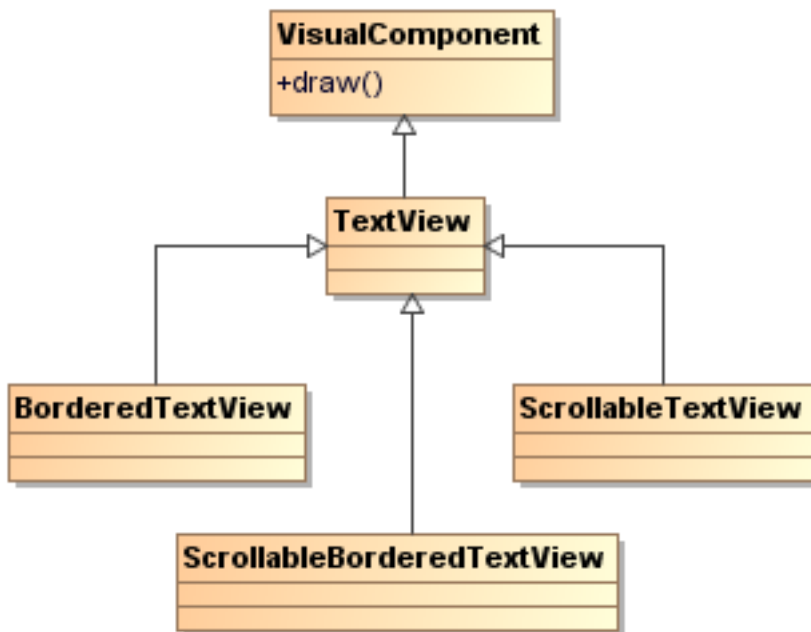
3.4.6 Демонстрација обрасца *Composite* на примеру у Јави

3.5 Decorator

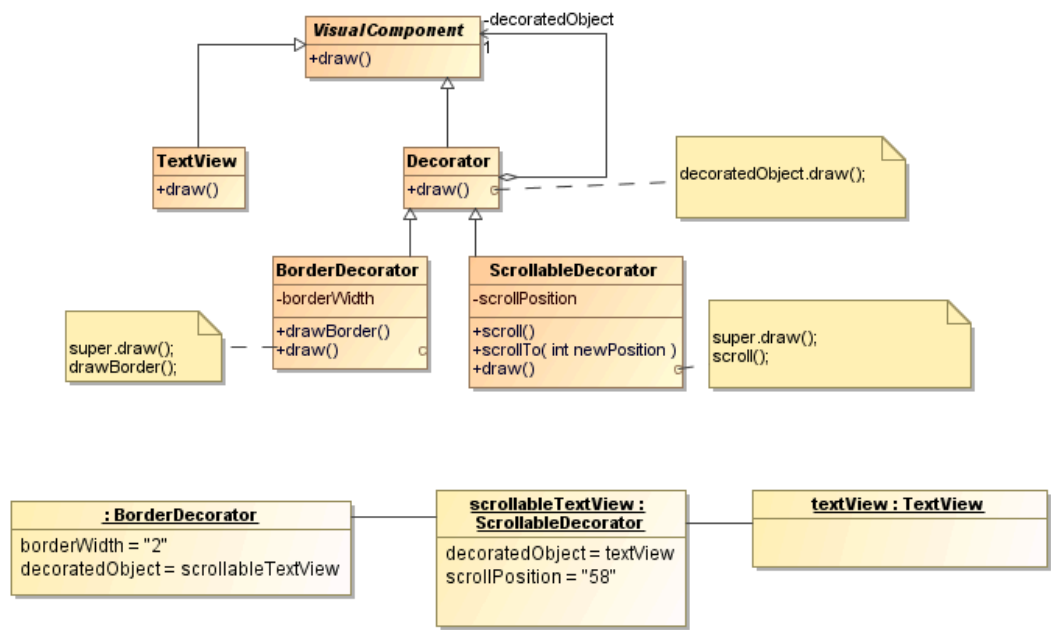
3.5.1 Decorator

- Додавање одговорности објекту динамички.
- Флексибилна алтернатива за наслеђивање.
- По структури сличан *Adapter* обрасцу али има другу намену.

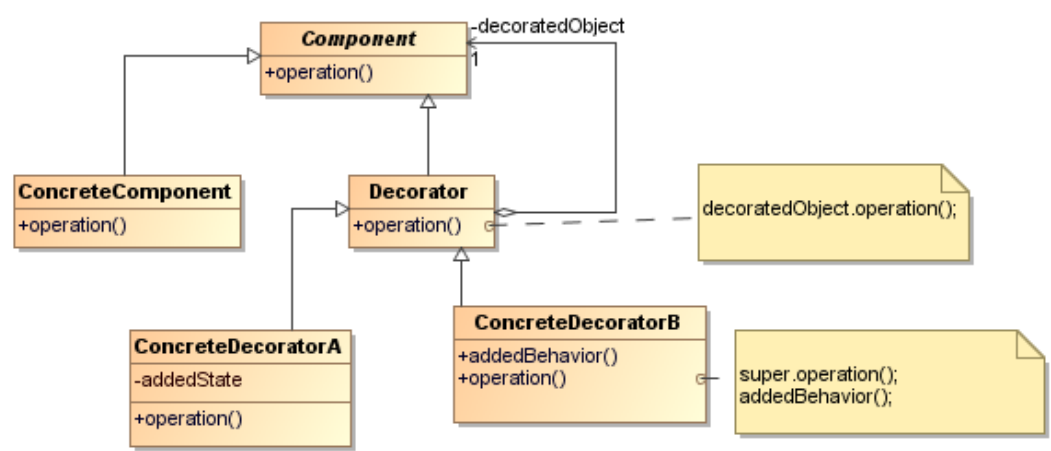
3.5.2 Мотивација



3.5.3 Пример



3.5.4 Структура обрасца



3.5.5 Пример - React Higher-Order Components (HOCs)

```
function withLogging(WrappedComponent) {
  return function(props) {
    console.log('Rendering:', WrappedComponent.name);
    return <WrappedComponent {...props} />;
  };
}

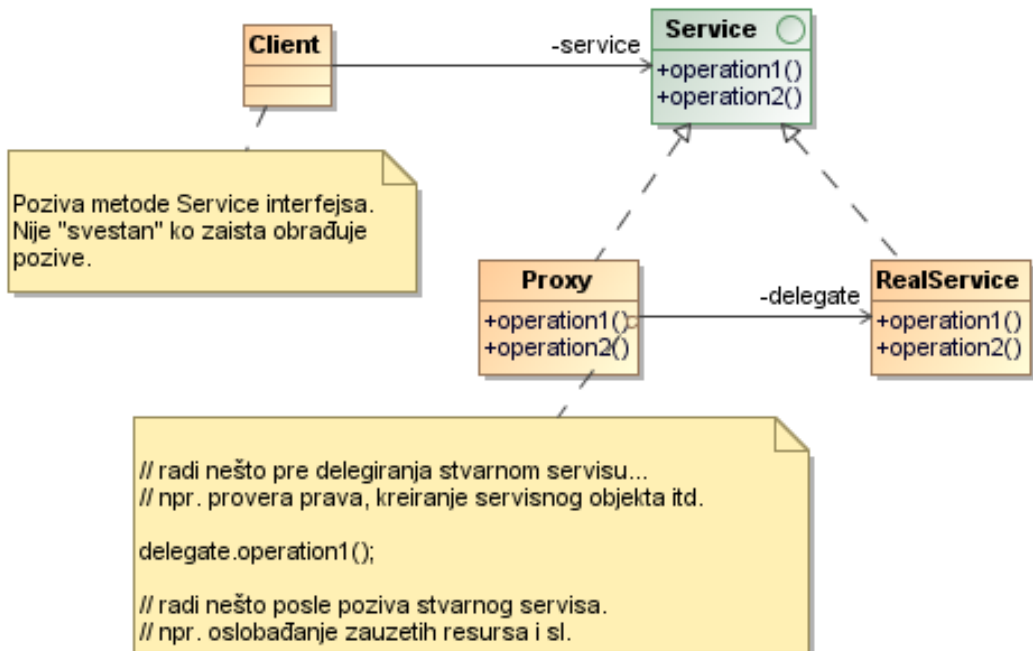
const EnhancedButton = withLogging(Button);
```

3.6 Proxy

3.6.1 Proxy

- Посредовање између објекта који пружа сервис и објекта који користи сервис у циљу имплементације контроле приступа, транспарентног приступа удаљеним објектима, АОР техника и сл.
- Имплементација најчешће делегира позиве метода објекту који пружа
- Клијент није свестан да постоји посредник између њега и објекта који обезбеђује сервис.
- Познат и под називом *Surrogate*.

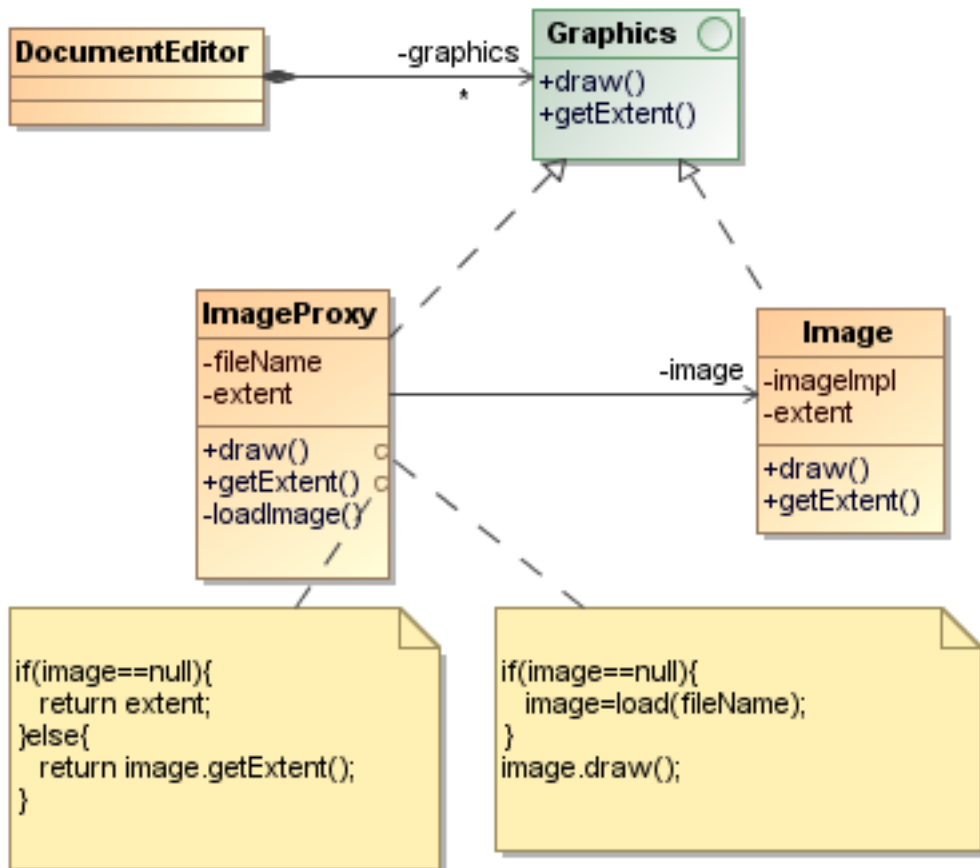
3.6.2 Структура обрасца



3.6.3 Када се користи?

- Одлагање скупе иницијализације објекта – виртуелни прокси (*Virtual Proxy*) или лењи прокси (*Lazy Proxy*).
- Приступ удаљеном објекту као да је локални (употребом технологија за дистрибуиране објекте - CORBA, DCOM, RMI и сл.) – *Remote Proxy*.
- Провера права приступа објекту – *Protective Proxy*.
- Синхронизација приступа (проверавање да ли је објекат закључан приликом приступа), бројање референци (*smart pointer*), имплементацију AOP техника – *Smart Proxy*.

3.6.4 Пример - *Lazy Proxy*



3.6.5 Copy-On-Write оптимизација

- Посебна врста лењег или виртуелног проксија.

- Одлагање копирања сложеног објекта (видети *Prototype* образац) за тренутак када се затражи операција модификације.
- Потребно бројање референци: свако креирање новог проксија увећава бројач референци, свако стварно копирање циљног објекта умањује бројач референци. Када бројач референци постане 0 објекат се може уклонити.

3.6.6 Демонстрација обрасца *Proxu* на примеру у Јави

3.7 Литература

- М. Grand, *Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML*, John Wiley & Sons, Inc., vol. 1, 2002
- Е. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994

Глава 4

Обрасци понашања

Садржај

4.1 Обрасци понашања

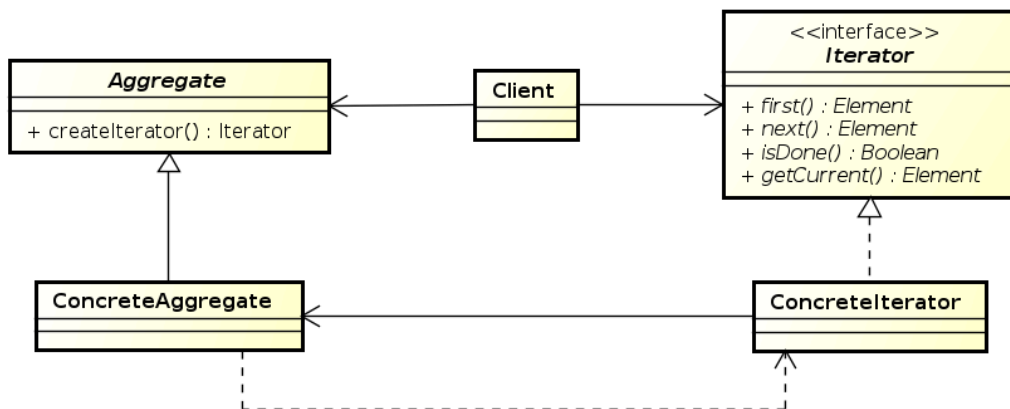
- Баве се алгоритмима и расподелом одговорности између објеката.
- Не дефинишу само образац структуре већ и образац комуникације између објеката.

4.2 Iterator

4.2.1 Iterator

Омогућава приступ елементима колекције секвенцијално без откривања конкретне репрезентације колекције.

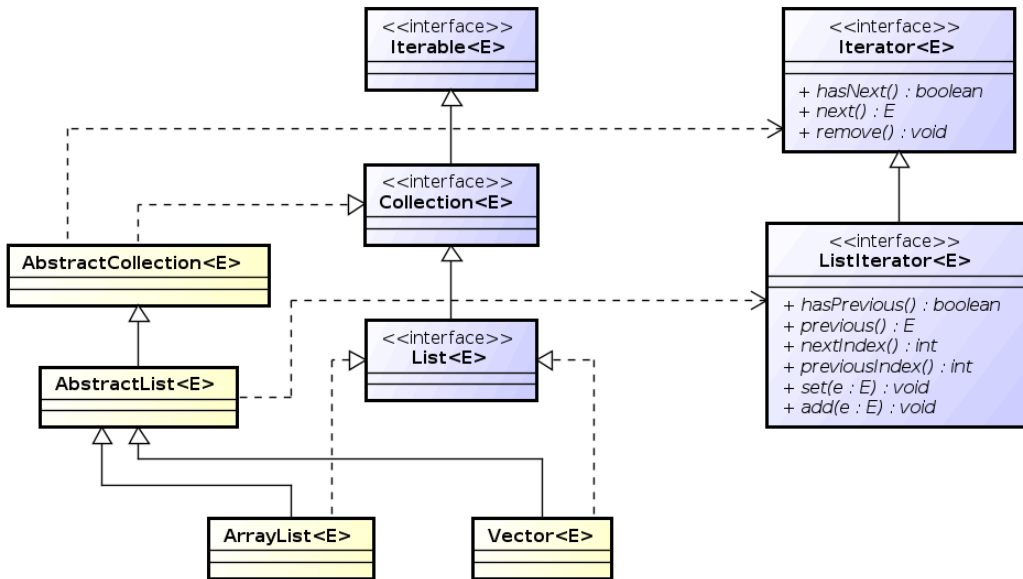
4.2.2 Структура обрасца



4.2.3 Јава имплементација

Итератор образац је директно подржан у јавиним стандардним библиотекама – интерфејси `Iterable<E>` и `Iterator<E>`.

4.2.4 Јава имплементација



4.2.5 Јава имплементација - While петља

```

List<Integer> intList = new ArrayList<Integer>();
Iterator<Integer> i = intList.iterator();
while(i.hasNext()){
    Integer a = i.next();
    System.out.println(a);
}

```

4.2.6 Јава имплементација - ForEach петља

Итератори су подржани директно у Јава програмском језику. *For-each* петља у јави се може користити за било који објекат који имплементира `Iterable<E>` интерфејс или за јаву низове.

```

List<Integer> intList = new ArrayList<Integer>();
for(Integer i: intList){
    System.out.println(i);
}

```

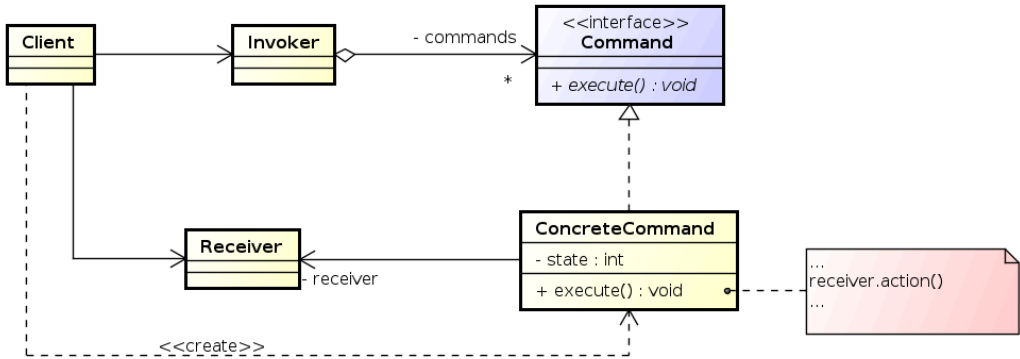
4.3 Command

4.3.1 Command

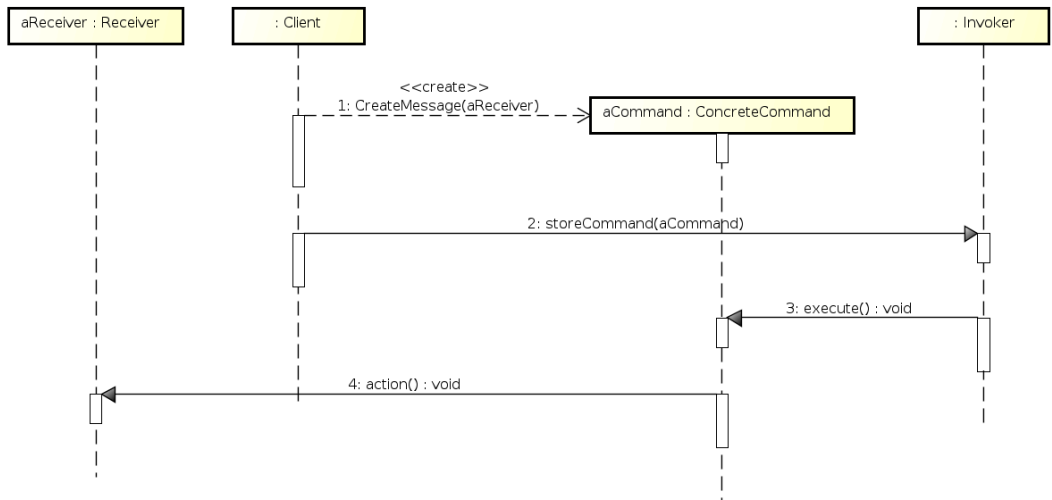
- Енкапсулација захтева за обрадом у виду објекта.
- Омогућава креирање редова за обраду (*Queues*) као и *undo* операције.

- Познат и под називима *Action* и *Transaction*.

4.3.2 Структура обрасца



4.3.3 Сарадња учесника



4.3.4 Шта добијамо?

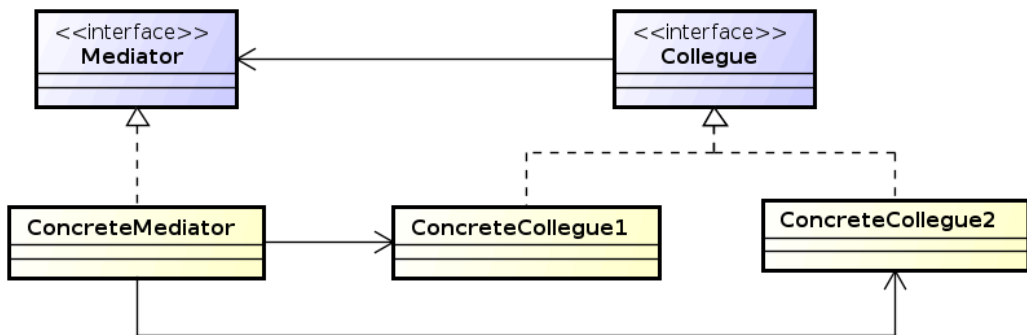
- Раздвајамо објекат који инницира извршавање операција од онога који “зна” како операцију treba извршити.
- *Command* објекти су објекти првог реда. Могу се третирати као и сви други објекти.
- Команде се могу компоновати (употребом *Composite* обрасца) и формирати сложене команде – макро команде.
- Нове команде се лако додају – није потребно изменити постојеће класе.

4.4 Mediator

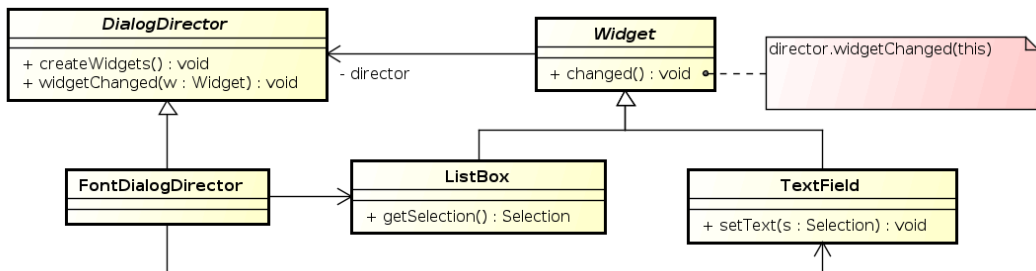
4.4.1 Mediator

- Објекат који енкапсулира знање о интеракцији групе објеката.
- Омогућава слабо спрезање (*loose coupling*) објеката тако што објекти не референцирају једни друге директно.

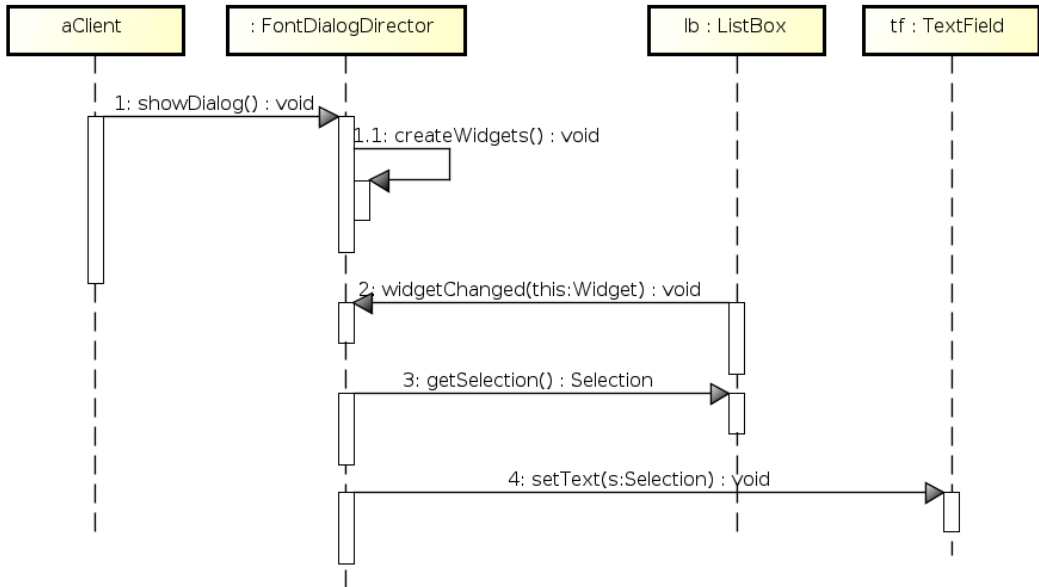
4.4.2 Структура обрасца



4.4.3 Пример



4.4.4 Сарадња учесника



4.4.5 Шта добијамо?

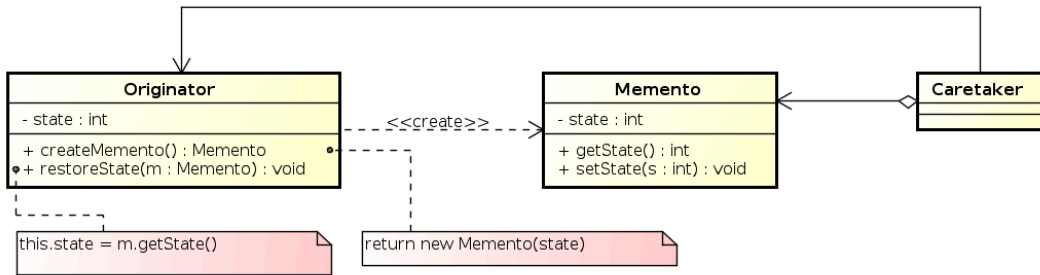
- Сложена интеракција објеката је централизована – измена се врши наслеђивањем једне класе.
- Објекти који ступају у интеракцију су слабо спрегнути – објекте и медијатор можемо мењати независно.
- Једноставнији протокол – везе *више-на-више* замењене везама *један-на-више* које су лакше за разумевање и измену.
- Логика интеракције објеката је одвојена од њиховог индивидуалног понашања – интеракција се лакше анализира.
- Проблем који може настати – монолитан и превише комплексан медијатор објекат.

4.5 Memento

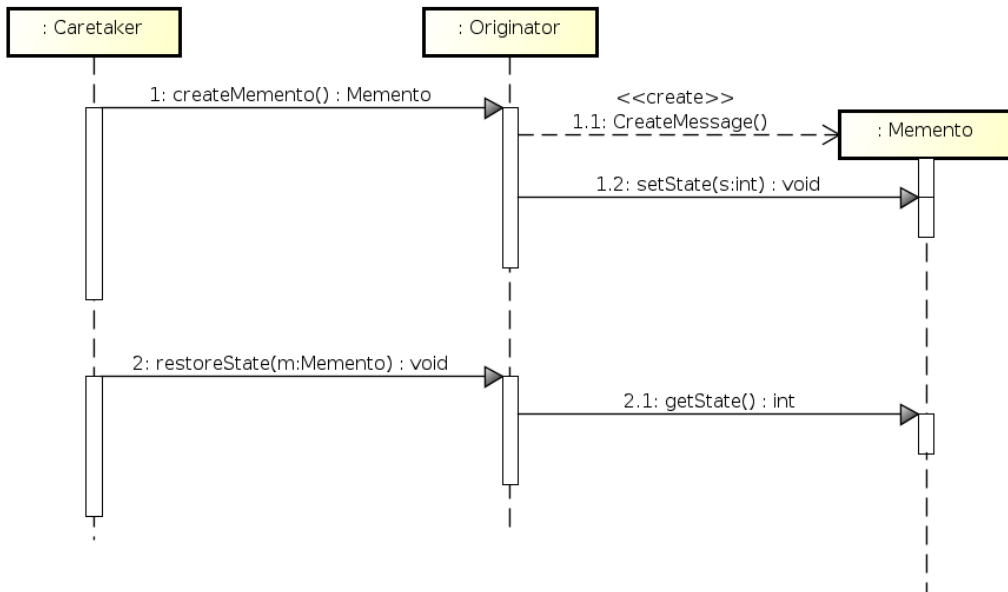
4.5.1 Memento

- Без нарушавања енкапсулације бележи и екстернализује интерно стање објекта тако да се објекат може касније вратити у идентично стање.
- Познат и под називом *Token*.

4.5.2 Структура обрасца



4.5.3 Сарадња учесника



4.5.4 Шта добијамо?

- Очувавамо енкапсулацију – мemento не открива детаље интерне имплементације *Originator* објекта иако се његово стање чува ван њега. Приступ *Memento* објекту је могућ само од стране *Originator* објекта.
- Поједностављен дизајн *Originator* објекта – клијенти чувају стање објекта кроз *Memento* објекат.
- Боља скалабилност - чување стања се препушта клијентима.
- Коришћење *Memento* објеката може бити проблематично уколико је стање *Originator*-а дефинисано великом количином података.

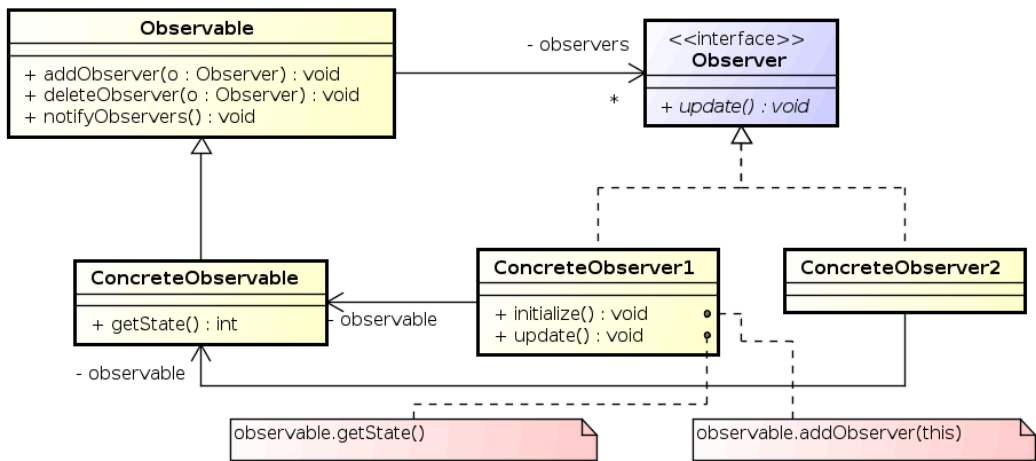
- У неким програмским језицима је тешко спречити приступ интерној структури *Memento* објекта од стране других објеката у систему.

4.6 Observer

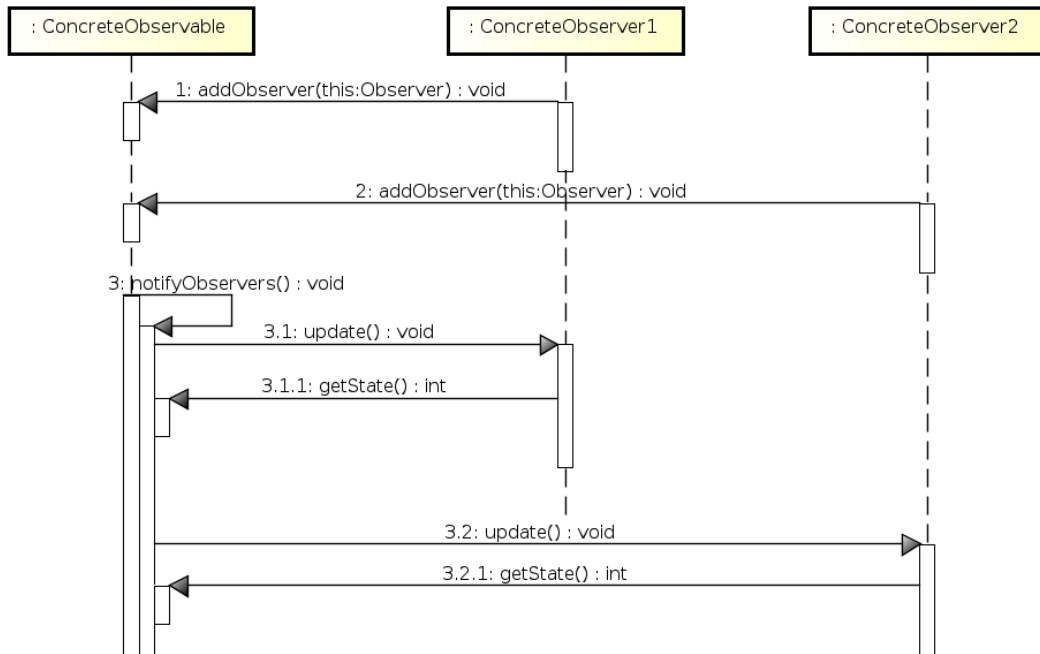
4.6.1 Observer

- Дефинише међузависност објеката тако да када се стање једног објекта промени сви зависни објекти се аутоматски обавештавају.
- Познат и под називом *Publish-Subscribe*.

4.6.2 Структура обрасца



4.6.3 Сарадња учесника



4.6.4 Подршка у Јави

- Интерфејс `Observer` и класа `Observable` у пакету `java.util`.
- Механизам `Listener`-а у `Swing`-у прати овај образац.

4.6.5 Шта добијамо?

- Апстрактно спрезање `Observer` и `Observable` објеката. `Observable` не зна конкретне класе `Observer` објеката већ комуницира са њима путем једноставног интерфејса.
- *Broadcast* стил комуникације. `Observable` јавља да се промена догодила. Не мора се наводити прималац поруке. Сви заинтересовани ослушкивачи ће добити нотификацију о промени.
- Уколико `Observable` не наводи у позиву поруке шта је промењено, `Observer`-и морају то сами да испитају што може бити “скупо”.

4.6.6 Напомене

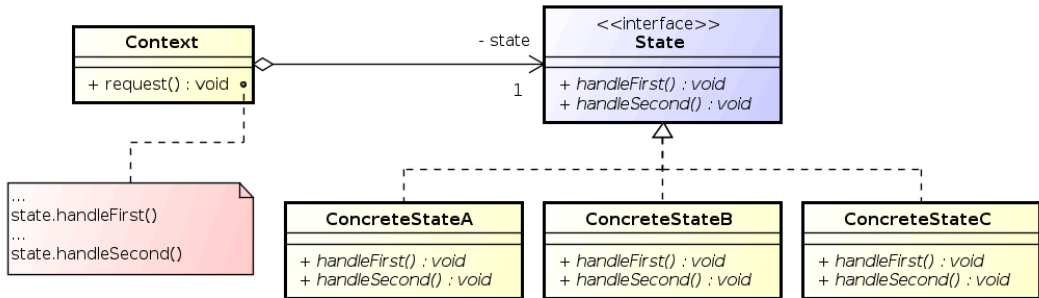
- Уколико `Observer` посматра више `Observable` објеката потребно је идентификовати објекат који шаље поруку о промени – слање референце на објекат као аргумент.
- Измена стања `Observable` објекта приликом обраде нотификације може довести до бесконачне рекурзије.

4.7 State

4.7.1 State

Измена понашања објекта приликом промене његовог интерног стања. Објекат се понаша као да је променио класу.

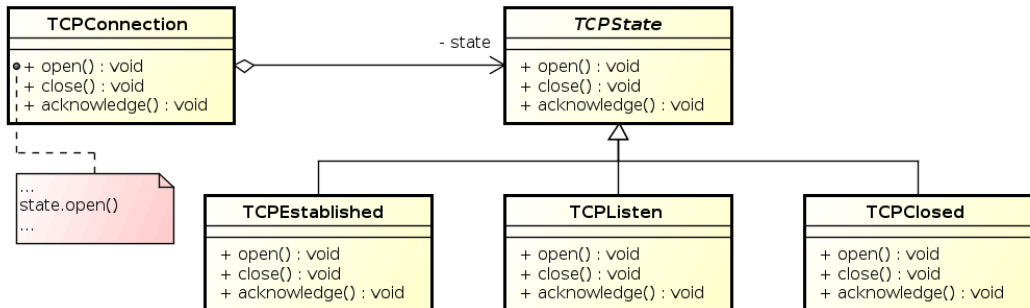
4.7.2 Структура обрасца



4.7.3 Сарадња учесника

- *Context* делегира операције зависне од стања објекту стате (*ConcreteStateX*).
- *Context* може проследити себе у захтеvu уколико стање треба да му приступа.
- Клијенти користе искључиво *Context*, мада могу, уколико је потребно, поставити текуће стање.
- Одлуку о преласку у ново стање може донети *Context* или текући стате објекат.

4.7.4 Пример



4.7.5 Шта добијамо?

- Локализација кода специфичног за одређено стање у једну класу - избегавање великих *switch/if-else* исказа.

- Експлицитан прелазак између стања.

4.7.6 Напомене

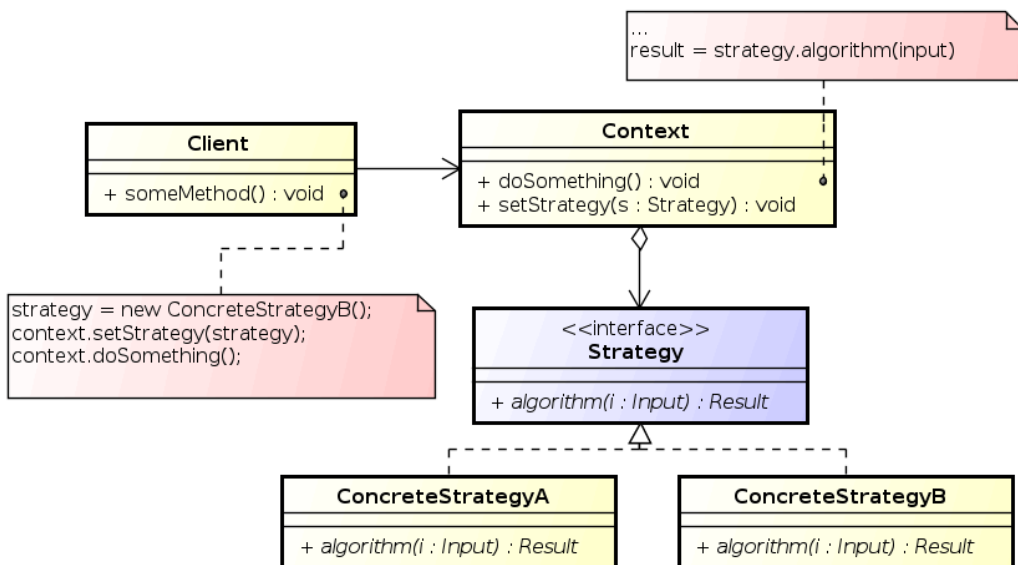
- Промена стања: *Context* или *ConcreteStateX*?
 - Уколико мења *ConcreteStateX* потребно је да приступа *Context*-у преко интерфејса за промену стања.
 - Негативно: стања морају да знају једна за друге.
 - Уколико *Context* мења стање – код може бити доста комплексан код већег броја стања.
- Креирање и уништавање *State* објеката: једном при иницијализацији или при свакој промени стања.
- Одређени програмски језици подржавају облик динамичког наслеђивања – природна подршка за *State* образац.

4.8 Strategy

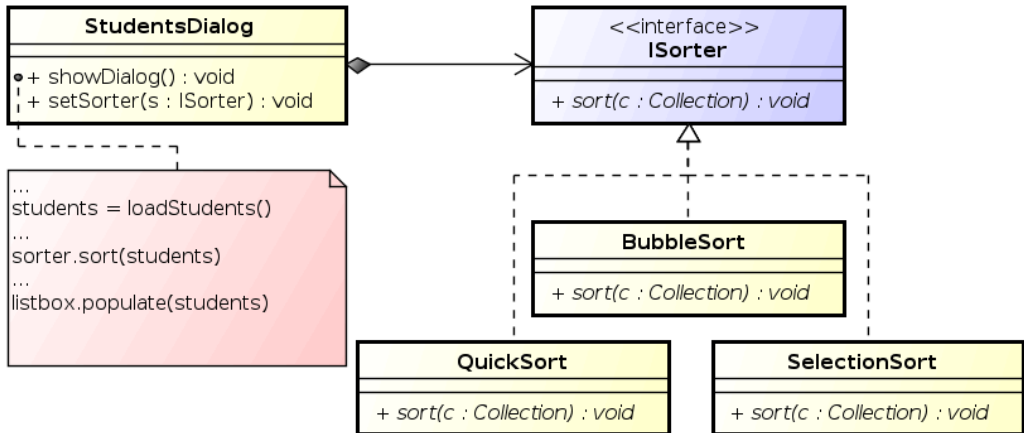
4.8.1 Strategy

Дефинисање фамилије алгоритама и омогућавање њихове измене без утицаја на клијента.

4.8.2 Структура обрасца



4.8.3 Пример



4.8.4 Шта добијамо?

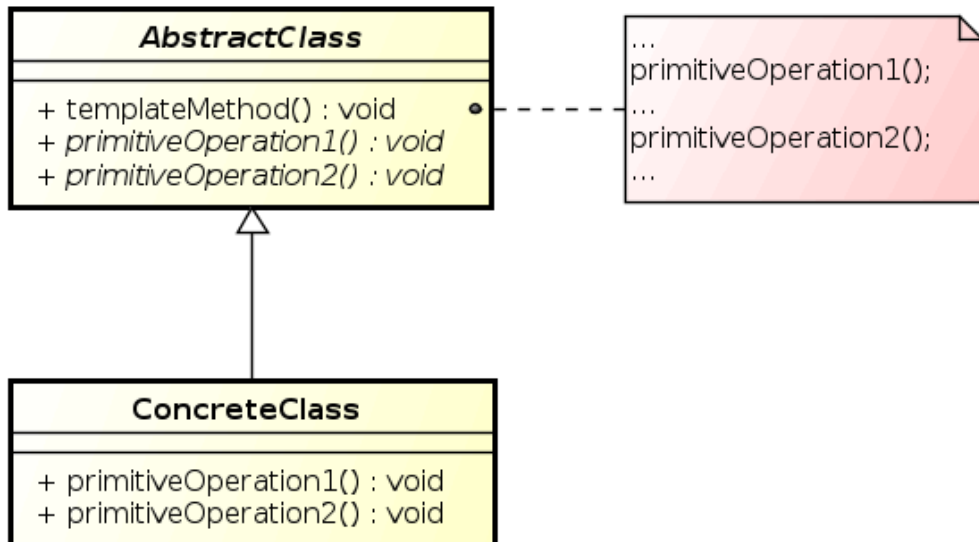
- Дефинисање фамилије алгоритама. Наслеђивање стратегија се може користити за имплементацију заједничке функционалности алгоритама.
- Измена понашања објекта динамички. Наслеђивањем контекст класе може се постићи измена функционалности али је она статичке природе.
- Елиминација исказа услова (*if-else*, *switch*).
- Омогућавамо клијенту избор алгорита и његових перформанси (нпр. мањи утросак меморије или брже извршавање).

4.9 Template Method

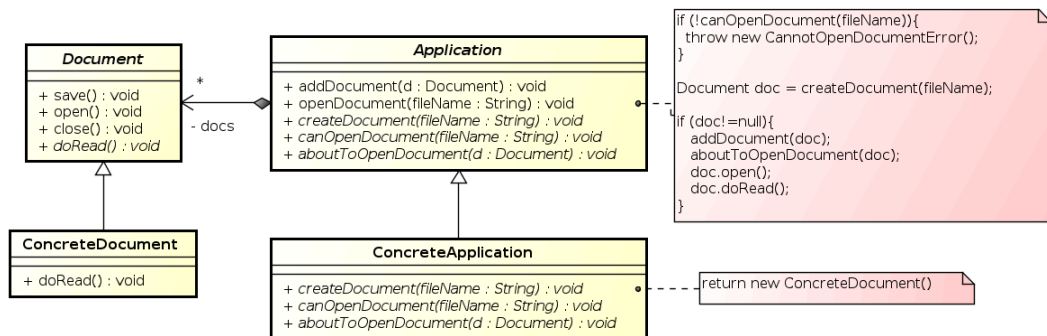
4.9.1 Template Method

- Дефинисање структуре алгорита при чему се дефинисање одређених корака оставља класама наследницама.
- Структура алгорита је непромењива али се одређени кораци могу променити.

4.9.2 Структура обрасца



4.9.3 Пример



4.9.4 Шта добијamo?

- Могућност дефинисања инваријантних делова алгорита у апстрактној класи а варијабилних делова у подкласама – спречава се дуплирање кода.
- Инвертовање контроле – *Hollywood principle*

Don't call us, we'll call you

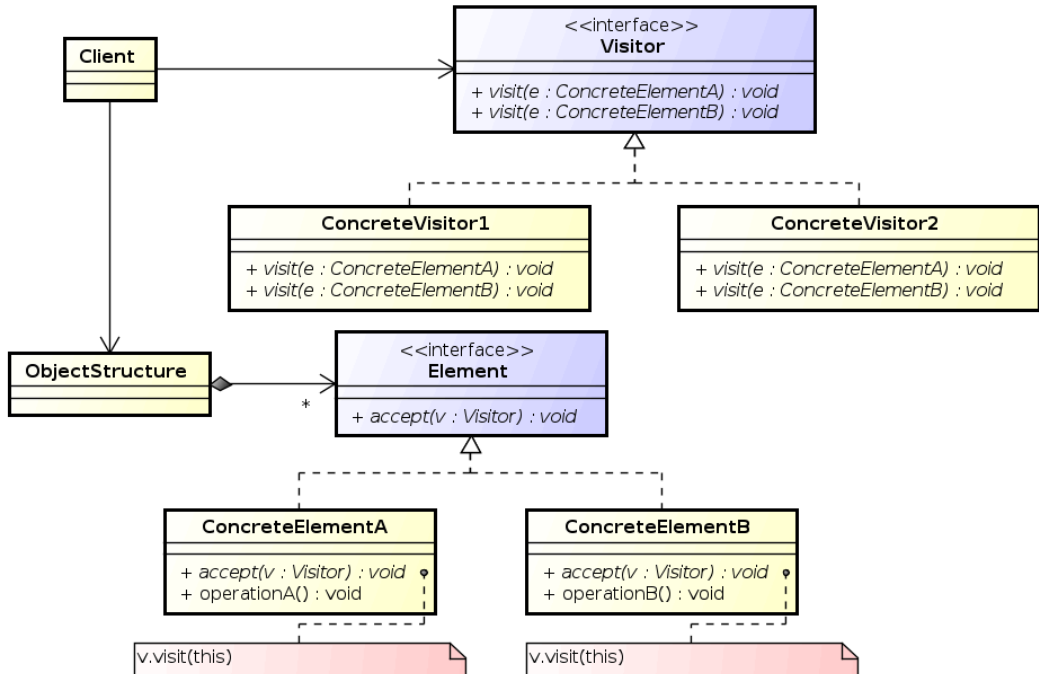
- Дефинисање тзв. *hook* операција. Најчешће су празне али могу имати дефинисано и стандардно понашање. За разлику од апстрактних операција, *hook* операције су опционе (не морају се редефинисати).

4.10 Visitor

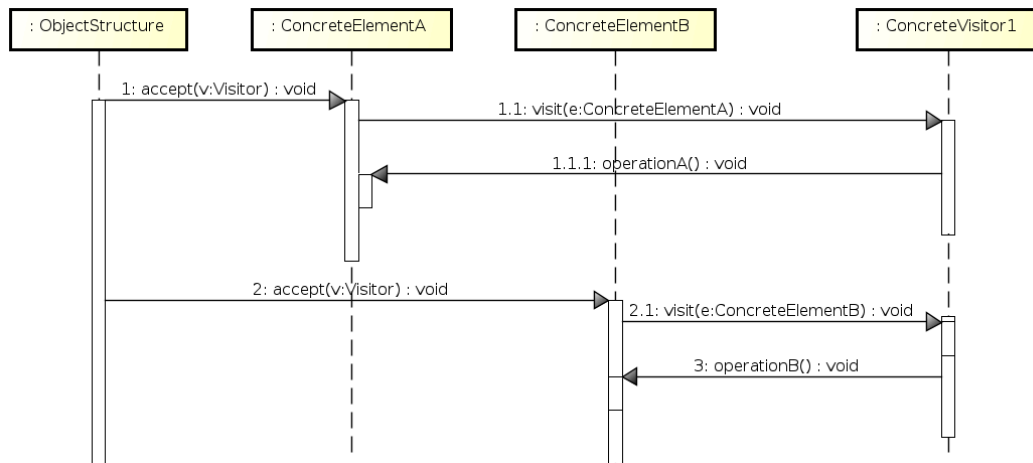
4.10.1 Visitor

- Раздвајање алгоритма од структуре података над којим оперише.
- Локализација имплементације операције која се извршава над елементима сложене структуре података (нпр. чворовима стабла).

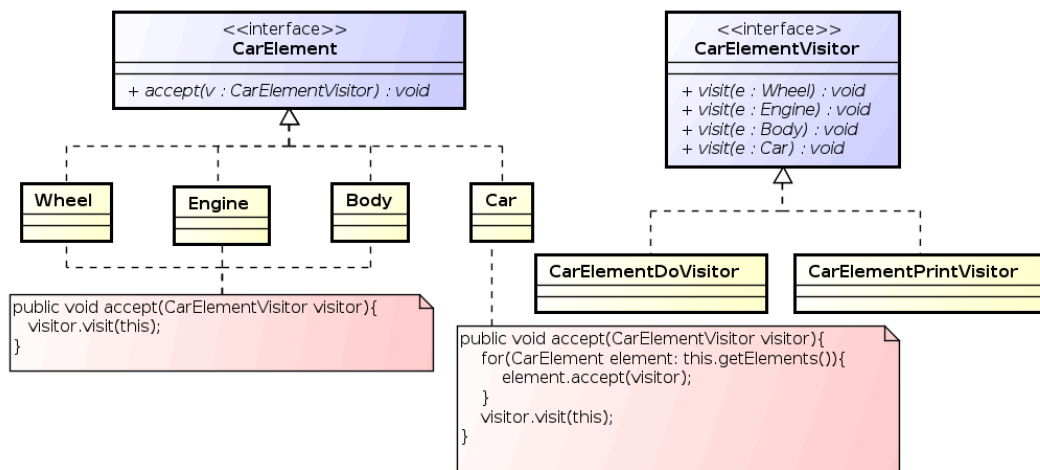
4.10.2 Структура



4.10.3 Сарадња учесника



4.10.4 Пример



4.10.5 Када користити?

- Објектна структура садржи елементе различитих класа а потребно је извршити операцију која зависи од конкретне класе објекта.
- Више различитих, несродних операција је потребно извршити над елементима сложене структуре а желимо избећи “загађивање”, свих класа чији објекти чине сложену структуру, са новом методом за сваку операцију.
- Хијерархија класа објектне структуре се ретко мења док додавање нових операција над структуром може бити честа операција.

4.10.6 Напомене

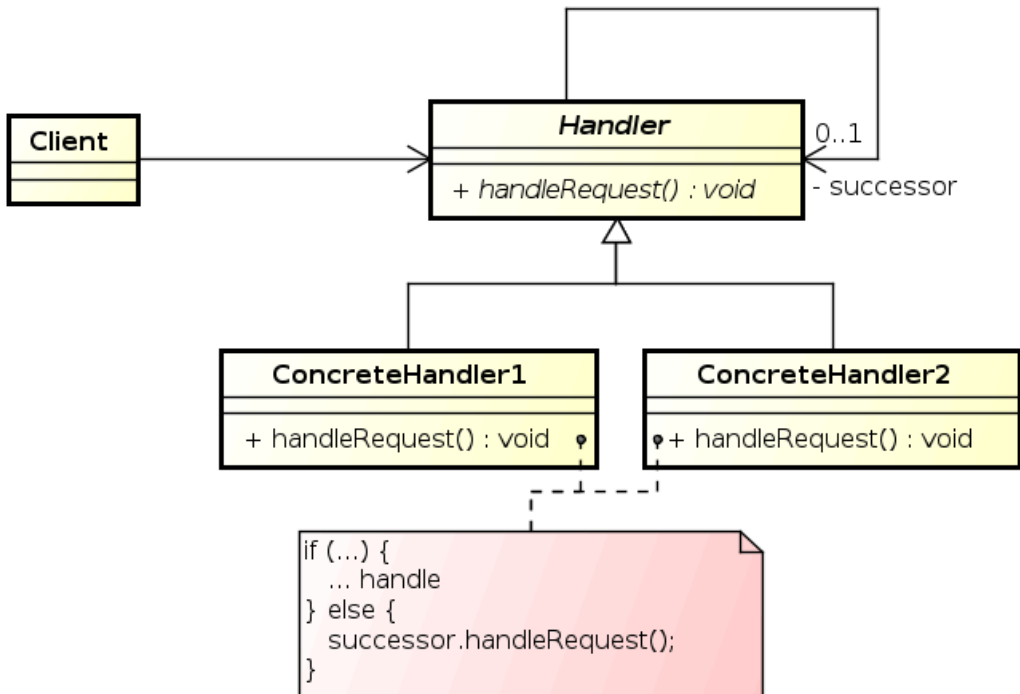
- *Visitor* представља имплементацију *Double Dispatch* механизма. Позвана операција зависи истовремено од типа конкретног *Visitor* објекта као и од типа конкретног елемента структуре над којом се операција врши.
- Сродне операције су локализоване у једној класи.

4.11 Chain of Responsibility

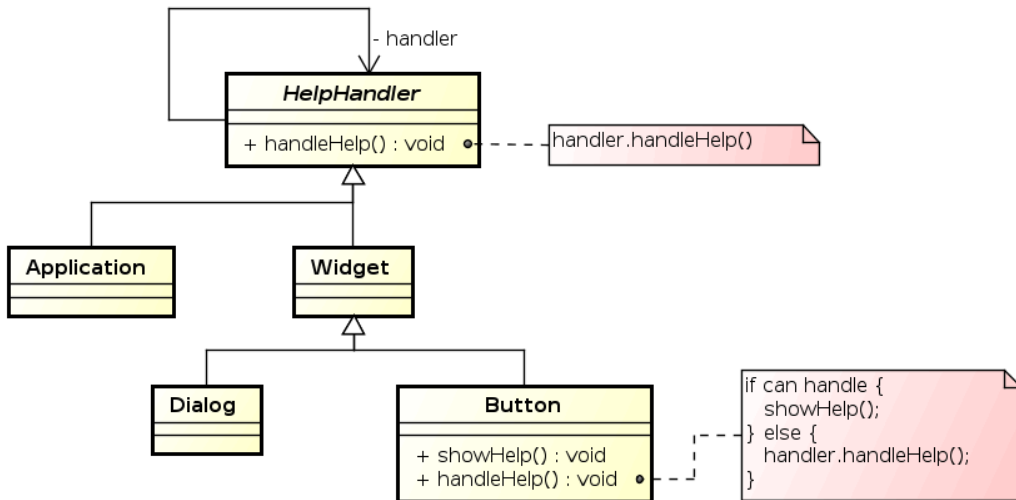
4.11.1 Chain of Responsibility

Избегавање јаког спрезања објекта који шаље захтев од објекта који врши обраду захтева тако што се објекти који врше обраду увезују у листу и даје се шанса сваком у низу да обради догађај све док неко не одговори позитивно на захтев (изврши обраду).

4.11.2 Структура



4.11.3 Пример



4.11.4 Када користити?

- Више од једног објекта може да обради захтев а обрађивач није познат унапред.
- Желимо да упутимо захтев једном из групе објеката а да не наводимо конкретног обрађивача експлицитно.
- Листа могућих обрађивача може динамички да се мења у време извршавања.

4.11.5 Напомене

- Две врсте: прослеђивање захтева низ ланац и после обраде или прекид прослеђивања.
- Објекат који врши обраду није експлицитно дефинисан па не постоји гаранција да ће захтев бити обрађен.

4.12 Литература

- E. Gamma, R. Helm, R. Johnson, and J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional, 1994
- M. Grand, *Patterns in Java: A Catalog of Reusable Design Patterns Illustrated with UML*, John Wiley & Sons, Inc., vol. 1, 2002

Глава 5

Микросервисни обрасци

Садржај

5.1 Увод

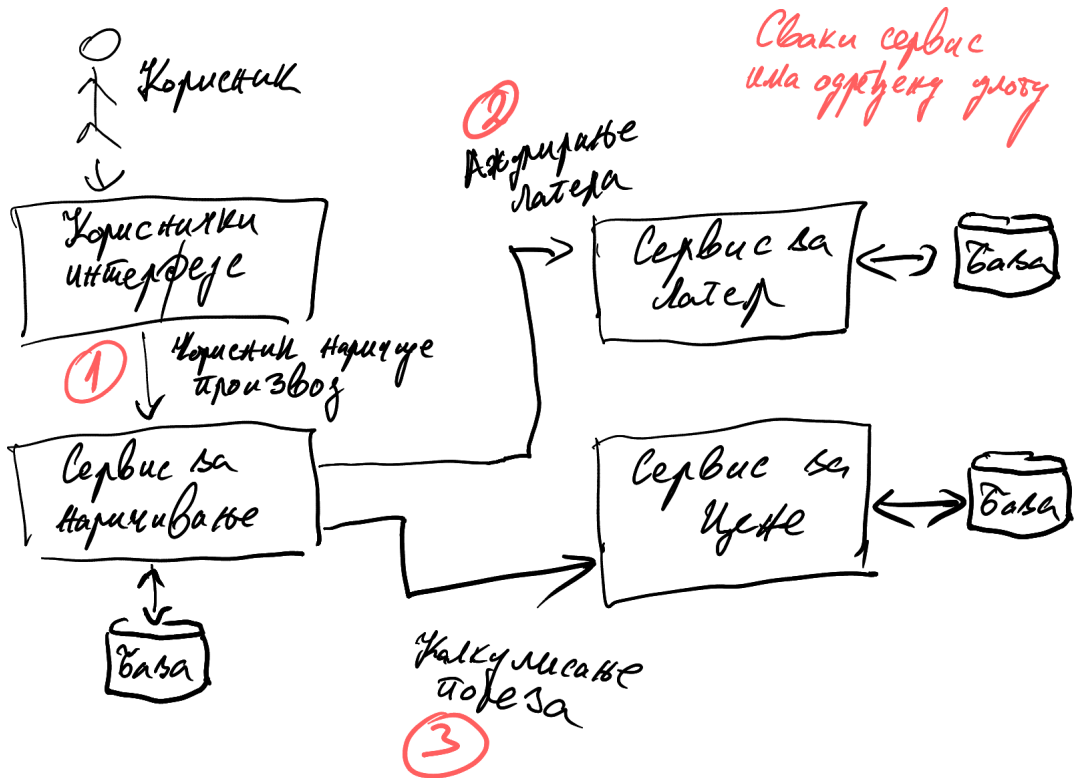
5.1.1 Микросервиси (*Microservices*)

- Софтверске компоненте.
- Висока кохезија, слаба спрега.
- Комуникација се обавља путем стандардних технолошки независних протокола (најчешће HTTP употребом REST стила уз JSON формат порука).
- Могу бити имплементирани у различитим програмских језицима и технологијама.
- Мали су, ограниченог контекста. Независно се развијају и распоређују.
- Имају јасно дефинисане интерфејсе путем којих комуницирају.
- *Unix* филозофија:

Do one thing and do it well.

5.1.2 Архитектура базирана на микросервисима

- Архитектонски стил где се апликација гради као скуп слабо спрегнутих “малих” сервиса (микросервиса) који сарађују.
- Варијанта *Service-Oriented Architecture* (SOA) али сервиси су “мали” и протоколи за комуникацију су једноставни (*light-weight*).
- Микросервиси пружају услуге и/или користе друге микросервисе.
- У циљу независне миграције микросервиса на нове верзије интерфејси се верзионирају и омогућава се клијентима да користе старе интерфејсе у прелазном периоду.



5.1.3 Предности у односу на монолитну архитектуру

- Тимови могу бити технолошки хетерогени.
- Распоређивање (*deployment*) се обавља у малим инкрементима (*fine-grained*).
- Модуларност, декомпозиција. Лакше разумевање, развој и тестирање. Отпорност на “ерозију архитектуре”.
- Боља скалабилност. Боља еластичност. Лако додавање нових микросервиса по потреби.
- Боља отпорност на отказе. Уколико један микросервис “падне” остатак апликације наставља да ради.
- Лакша миграција на нове технологије. Могућа постепеним заменама микросервиса.
- Интеграција хетерогених и “старих” система (*legacy*).
- Континуална интеграција и достава (*Continuous Integration/Delivery*)

5.1.4 Мане у односу на монолитну архитектуру

- Више “покретних делова”. Захтева боље алате за распоређивање и надзор.

- Теже дебаговање. Дебаговање захтева праћење захтева кроз више микросервиса који се извршавају често на различитим физичким/виртуелним рачунарима.
- Додатни трошкови (*overheads*) услед комуникације.
- Додатни трошкови у случају потребе за дељењем података.

5.2 Обрасци за рад са базама података

5.2.1 База података по сервису (*Database per service*)

- У циљу слабог спрезања сервиса подаци над којима сервиси оперишу се имплементирају као приватни.
- Други сервиси не могу приступити подацима директно већ само кроз интерфејс сервиса.

<https://microservices.io/patterns/data/database-per-service.html>

Структура

```
cloud "клијенти" as klijenti
component "Сервис за наручивање" as servis1
database baza1 [
```

Наруџбенице

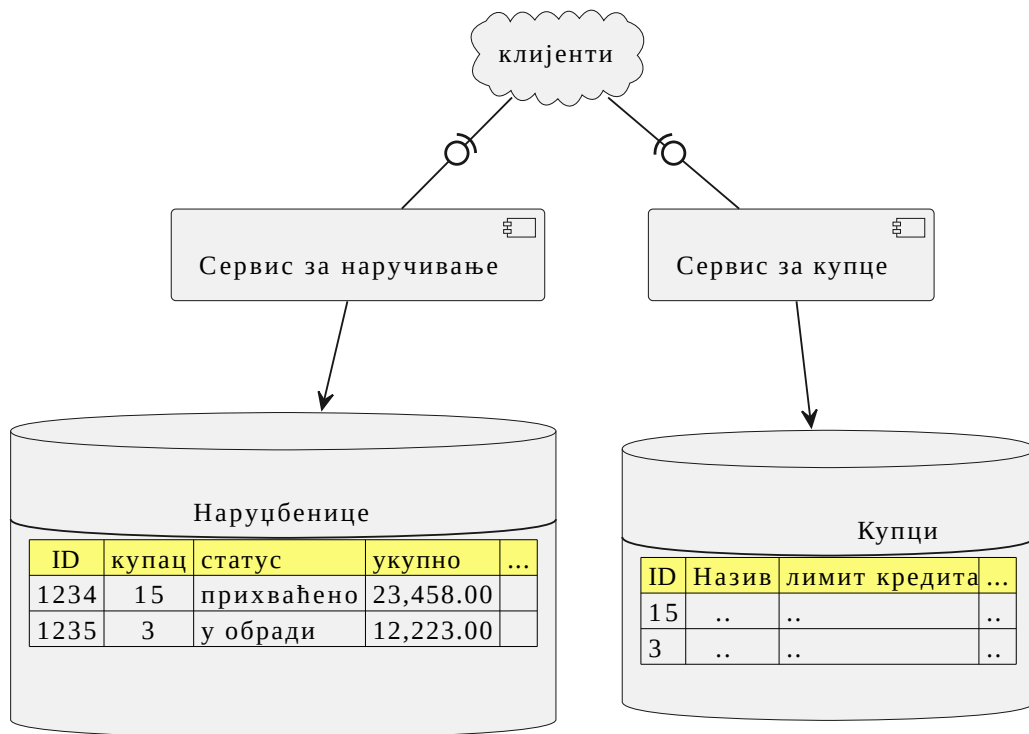
```
----
<#FBFB77>| ID | купац | статус      | укупно      | ... |
| 1234 | 15 | прихваћено | 23,458.00 |      |
| 1235 | 3  | у обради  | 12,223.00 |      |
```

```
]
servis1 --> baza1
servis1 -up0)- klijenti
component "Сервис за купце" as servis2
database baza2 [
```

Купци

```
----
<#FBFB77>| ID | Назив | лимит кредита| ... |
| 15 | .. | .. | .. | .. |
| 3  | .. | .. | .. | .. |
```

```
]
servis2 --> baza2
servis2 -up0)- klijenti
```



Предности

- Подаци су део имплементације сервиса.
- Имплементација приватне базе се може мењати независно од остатка система.
- Могуће је користити хетерогене технологије.

Мане

- Отежано извођење трансакција које се обављају између података различитих сервиса. Видети образац [Saga](#).
- Отежани сложени упити који обухватају податке више сервиса. Видети образац [CQRS](#).

Начини имплементације

- Употреба једне инстанце базе за све сервисе:
 - приватне табеле по сервису,
 - приватна шема по сервису.
- Посебна инстанца базе по сервису.

5.2.2 Дељена база података (*Shared database*)

- У циљу подршке за ACID трансакције сервиси деле исту базу података и могу слободно да приступају подацима других сервиса.

<https://microservices.io/patterns/data/shared-database.html>

Структура

```
cloud "клијенти" as klijenti
component "Сервис за наручивање" as servis1
database "Јединствена база" {

  card narudzbenica [

                                Наружбенице
                                ----
                                <#FBFB77>| ID | купац | статус      | укупно      | ... |
                                | 1234 | 15 | прихваћено | 23,458.00 |      |
                                | 1235 | 3  | у обради  | 12,223.00 |      |

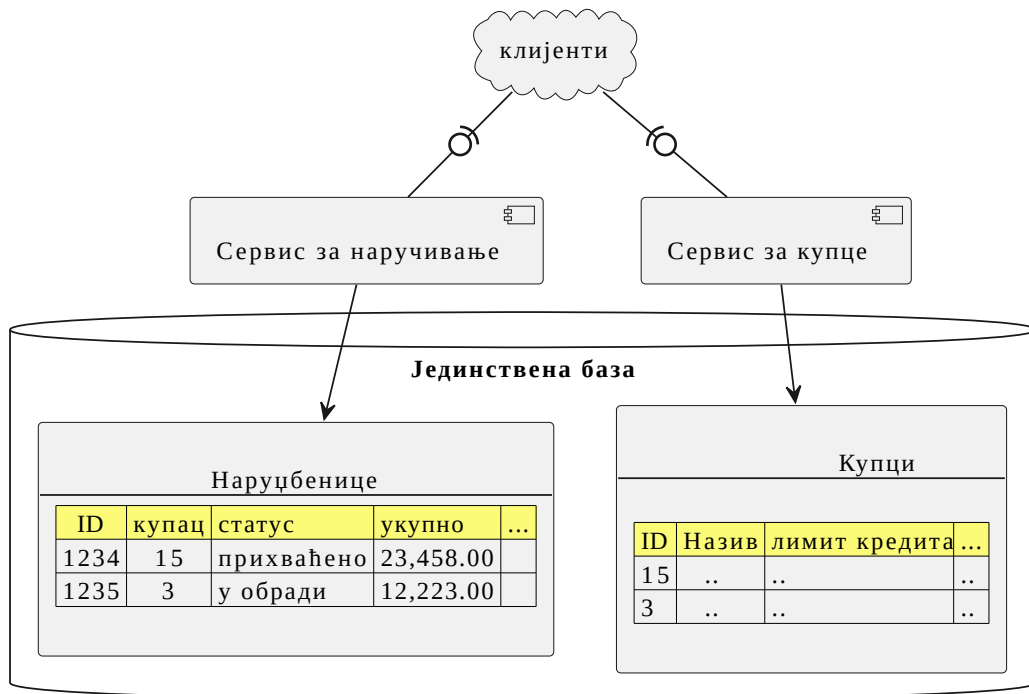
                                ]

  card kupci [

                                Купци
                                ----
                                <#FBFB77>| ID | Назив | лимит кредита| ... |
                                | 15 | .. | .. | .. | .. |
                                | 3  | .. | .. | .. | .. |

                                ]

}
servis1 --> narudzbenica
servis1 -up0)- klijenti
component "Сервис за купце" as servis2
servis2 --> kupci
servis2 -up0)- klijenti
```



Предности

- Једноставније за имплементацију и операцију.
- Једноставније трансакције (ACID) и упити (нпр. могућ JOIN између табела различитих сервиса).

Мане

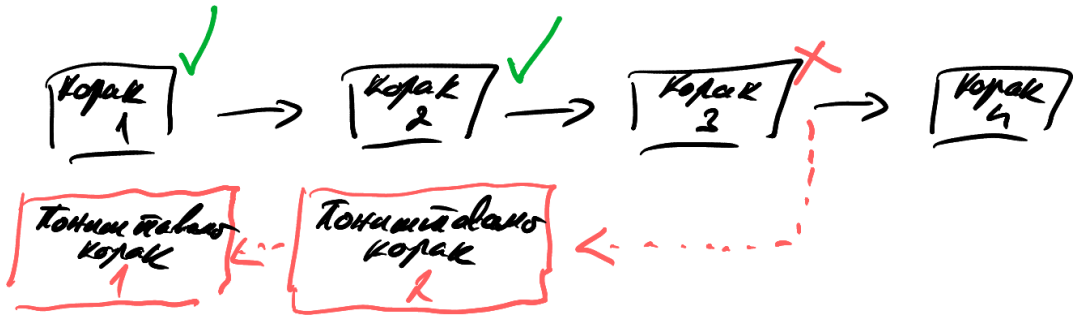
- Јача спрега између сервиса у време развоја (нпр. одржавање шеме базе мора бити координисано између тимова) и у време извршавања (нпр. један сервис може да закључа табелу и спречи друге сервисе да приступе).
- Сервиси могу да мењају податке других сервиса. Ово је могуће административно регулисати уколико база подржава.
- Иста база можда неће задовољити потребне функционалне и нефункционалне особине захтеване од стране неких сервиса.

5.3 Обезбеђивање конзистенције

5.3.1 Saga

- Вид дистрибуиране трансакције. Мање ригидна од *two-phase commit* (2PC).

- Очување конзистенције података између сервиса.
- Користи се када је у употреби [Database per service](#) образац за имплементацију трансакција.
- Низ локалних трансакција (*ACID*) које објавом поруке/догађаја иницирају следећу трансакцију у ланцу. Уколико нека од трансакција не успе, извршава се поништавање.

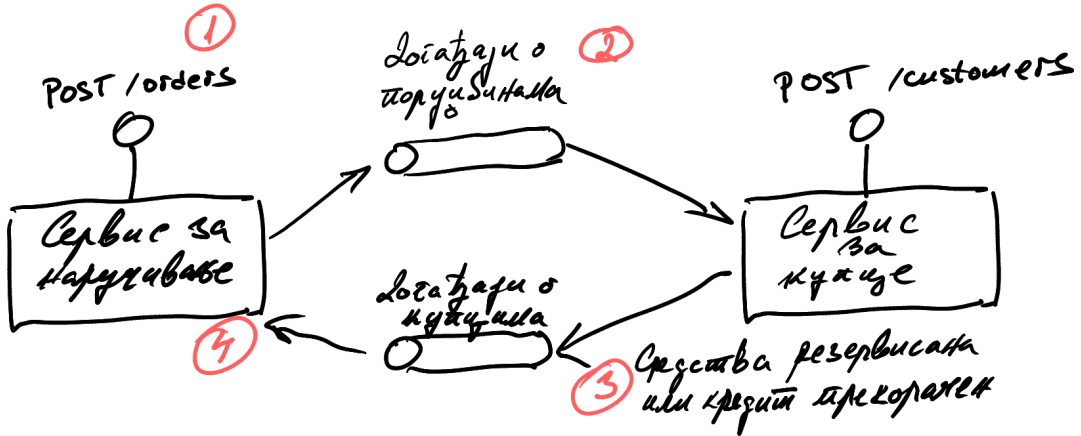


Приступи

- Два приступа у координацији трансакције:
 - Базиран на кореографији (*Choreography*) - после сваке локалне трансакције објављује се догађај који иницира извршавање следеће трансакције у низу.
 - Базиран на оркестрацији (*Orchestration*) - оркестратор (посебан објекат) је задужен да обавести учеснике да започну или да пониште трансакцију.

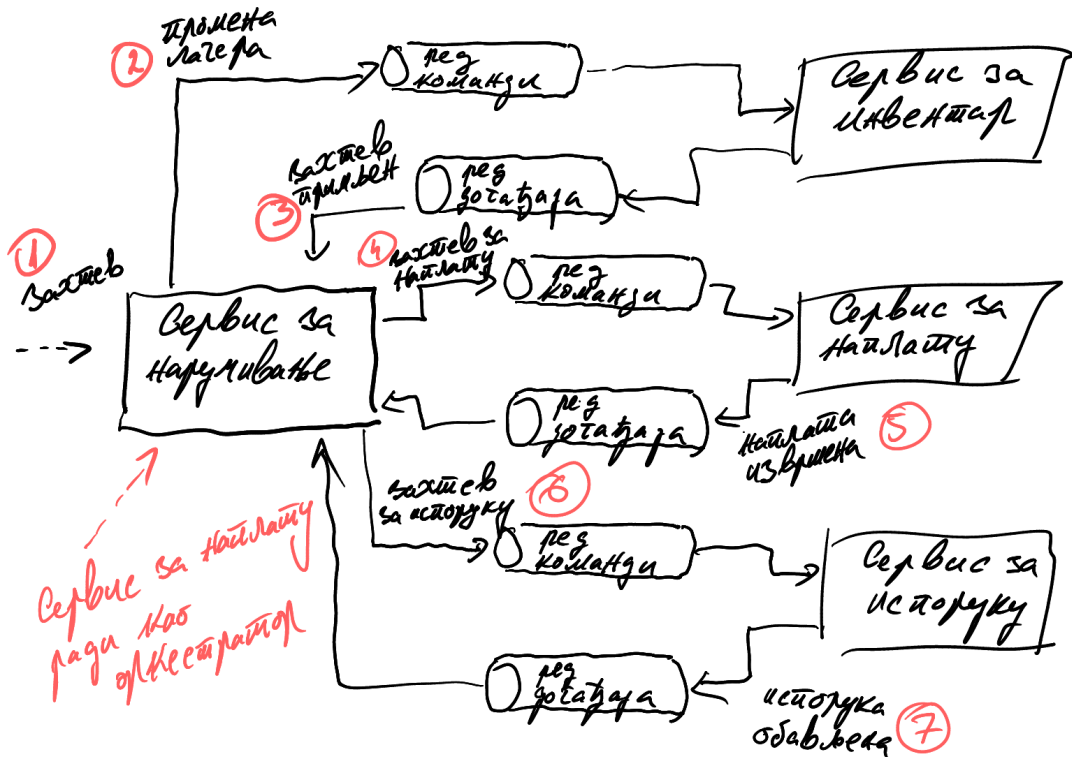
Структура - кореографија

1. Сервис за наручивање прима захтев `POST /orders` креира наруџбину у стању у обради
2. Прослеђује догађај Наружбина креирана
3. Сервис за купце покушава да резервише средства и објављује догађај који представља резултат операције: средства су резервисана или је кредит прекорачен.
4. Сервис за наручивање прихвата или одбија наруџбину.



Структура - оркестрација

- Сервис за наручивање је оркестратор тј. задужен је за координацију целокупног процеса наручивања.



Предности

- Слабија спрега. Мање ригидан систем.

- Боља скалабилност.

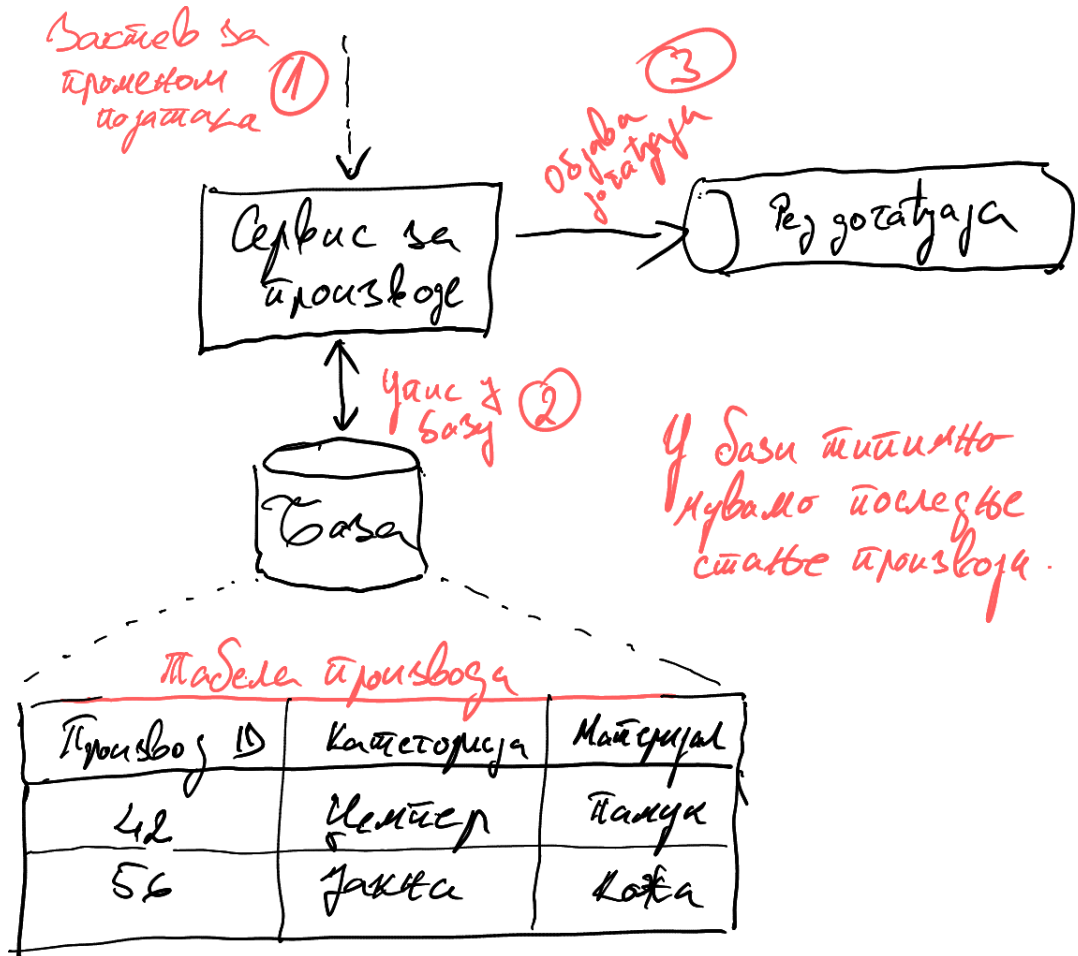
Мане

- Мора се пажљиво моделовати редослед операција због потенцијалног поништавања.
- Шта радити у ситуацији када је операција неуспешна због техничког проблема (нпр. сервис није тренутно доступан)?
- Модел конзистентности - коначна конзистентност (*Eventual Consistency*) - немамо гаранције да ће прочитани подаци бити право стање система тј. да ће бити конзистентни.
- BASE семантика (*Basically-Available, Soft-state, Eventual consistency*)
 - *Basically-Available* - добра доступност података али без гаранције на конзистентност
 - *Soft-state* - одређена вероватноћа да знамо стање јер систем можда још није конвергирао.
 - *Eventual consistency* - ако изменимо стање после одређеног времена измена ће бити видљива свим клијентима.

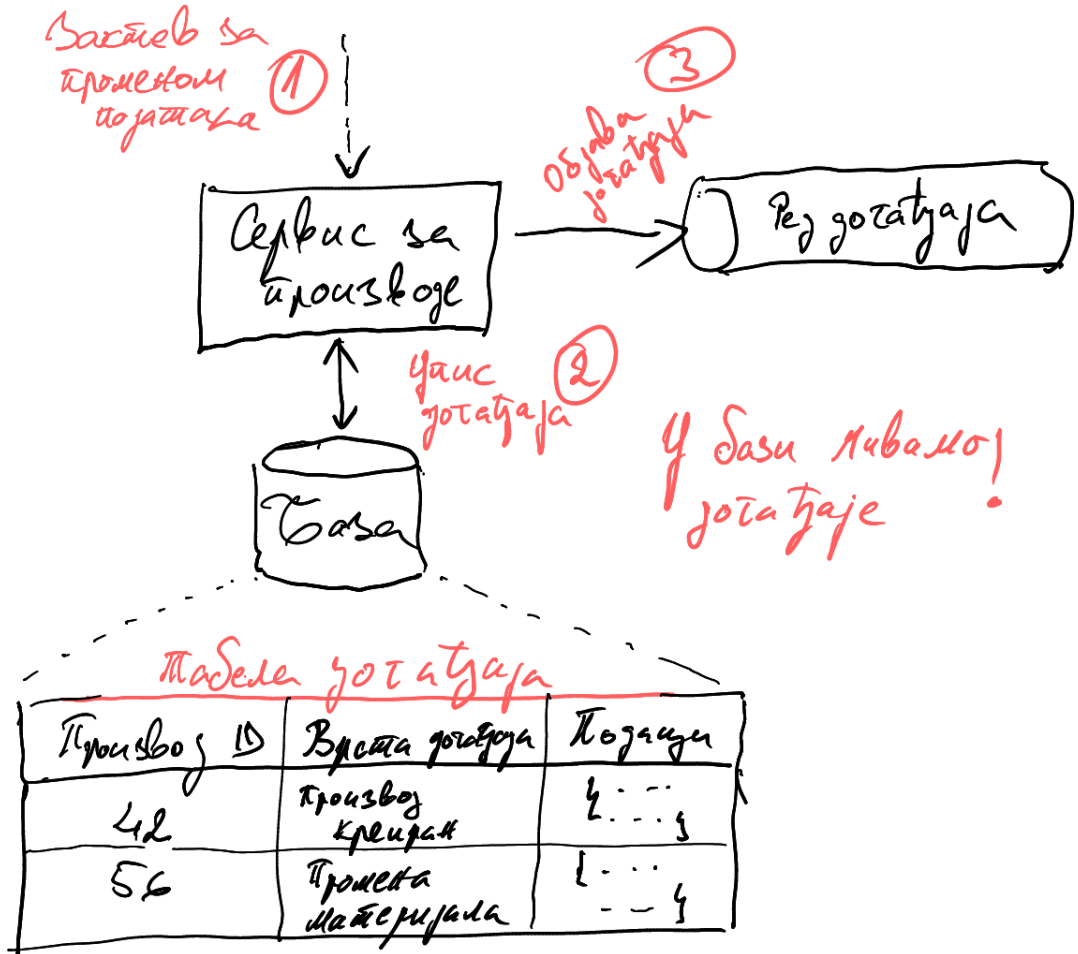
5.3.2 Event sourcing (ES)

- Користи се код архитектуре вођене догађајима (*Event-Driven*).
- Уместо чувања тренутног стања ентитета чува се низ догађаја који су мењали ентитет.
- Текуће стање се може добити применом свих догађаја до садашњег тренутка.
- Природно се користи са обрасцем [CQRS](#).

Структура код класичног приступа



Структура код ES приступа



Напомене

- Ток догађаја треба да буде једини извор текућег стања.
- Предности:
 - различити модели се могу изградити применом тока догађаја у будућности.
 - природно садржи пуну историју измена што омогућава ревизију и контролу.
- Мана: немогућност постављања упита над током догађаја - због тога се користи у синергији са [CQRS](#).
- Додатно можемо користити брокере порука (*message brokers*) уместо базе података.

5.4 Постављање упита

5.4.1 API композиција (API Composition)

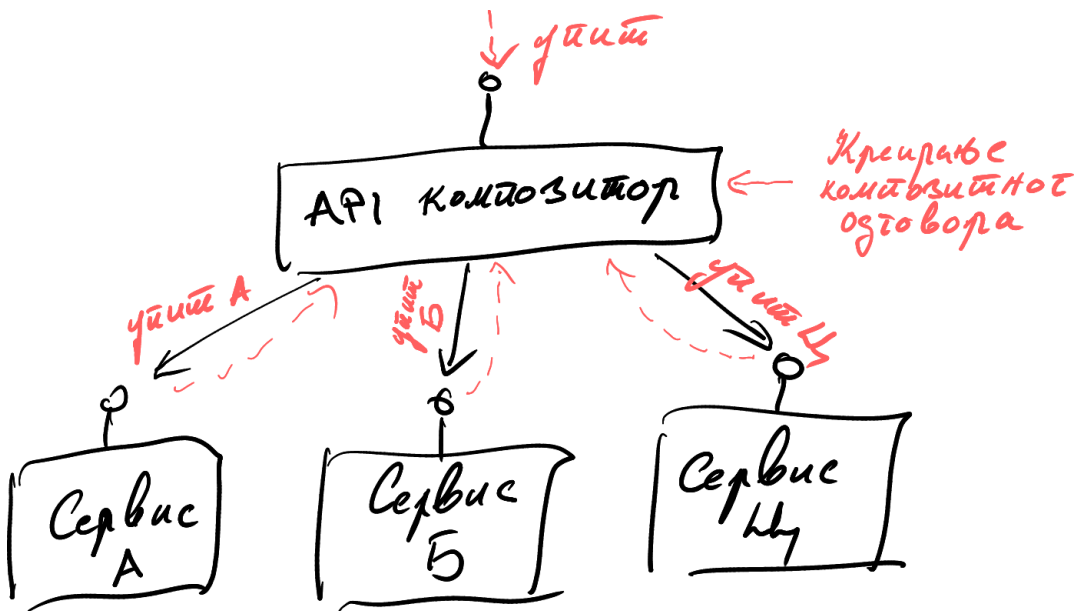
- У случају употребе обрасца *Database per service* поставља се питање како имплементирати упите који удружују податке из различитих микросервиса.

<https://microservices.io/patterns/data/api-composition.html>

Решење

- Креирати сервис који поставља појединачне упите и затим комбинује податке у меморији и враћа јединствени одговор са удруженим подацима.

Структура



Предности и мане

- Предности:
 - Поједностављење сложених упита.
 - Једно место за ажурирање сложених упита.
- Мане:
 - Поједини упити могу бити неефикасни јер се велика количина података преноси преко мреже и удружује у меморији.

Пример

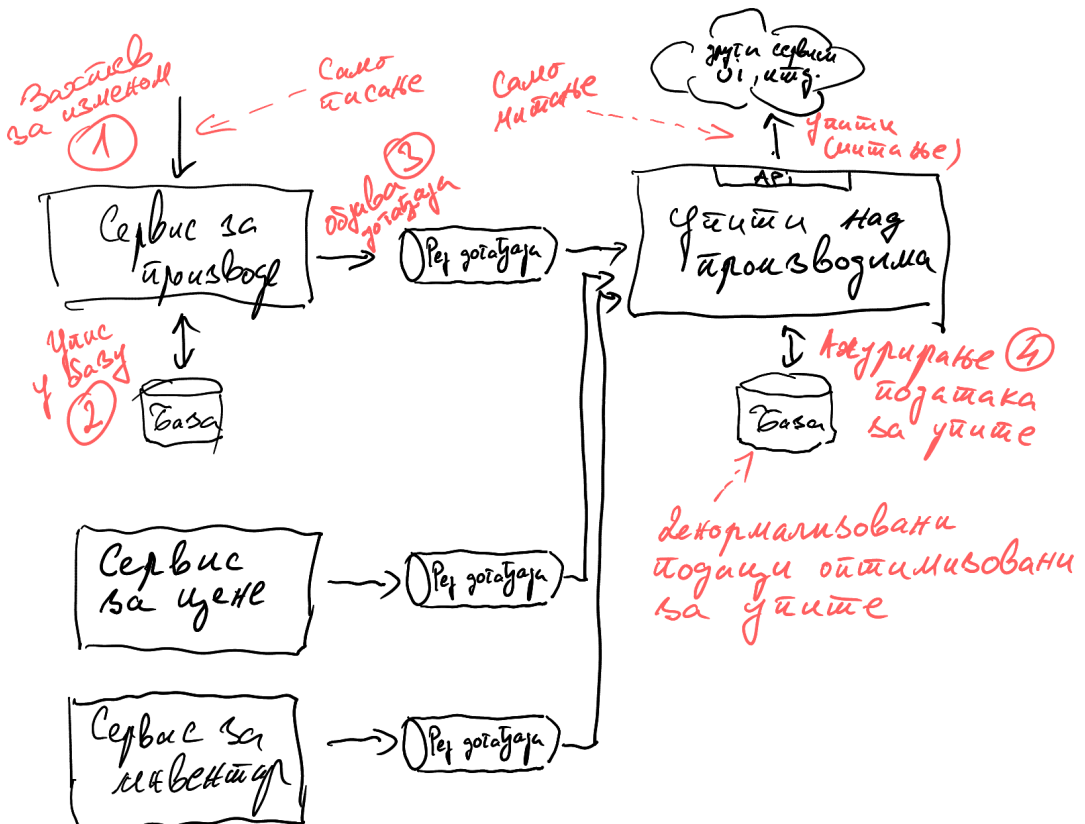
- Образец *API Gateway* често ради API композицију.

5.4.2 CQRS

- *Command Query Responsibility Segregation (CQRS)* се користи у ситуацији када имамо обрасце *Database per service* и *Event sourcing* имплементирани и желимо да подржимо упите који удружују податке из више микросервиса.
- Базиран на идеји поделе захтева на оне који мењају стање и оне који само читају тј. немају бочне ефекте. Еквивалентно са *REST* методама за читање (GET, HEAD) и измену стања (POST, PUT, PATCH...).
- Креирање базе која је само за читање и која се континуално ажурира обрадом догађаја који се емитују при промени података.

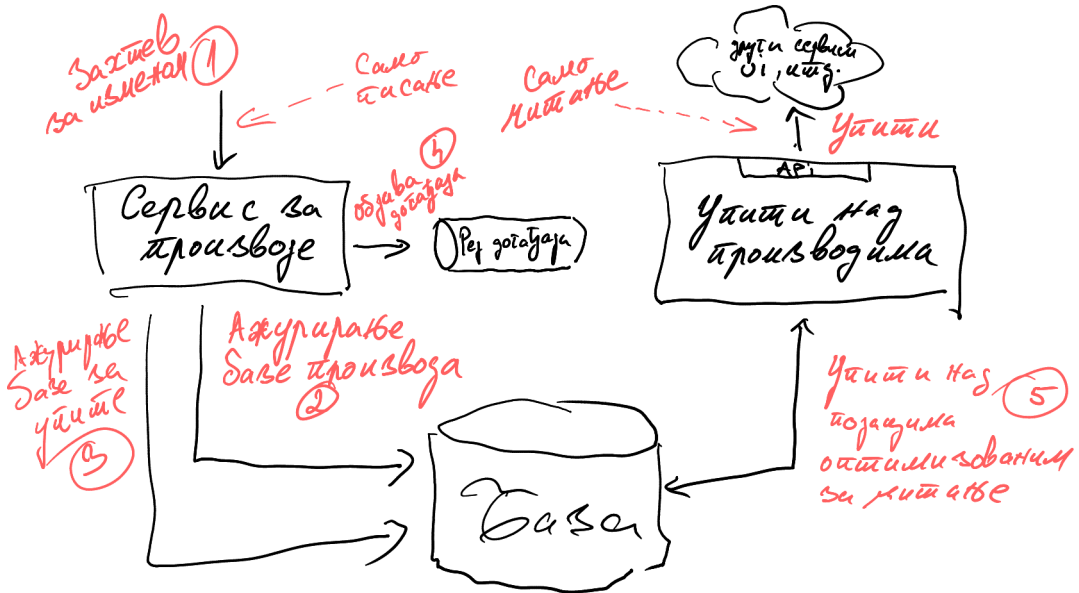
Структура - посебне базе

- Захтев за изменом производа (1) уписује текуће стање у локалну базу (2) и објављује догађај (3).
- Сервис за упите, на основу догађаја, ажурира (4) свој интерни модел оптимизован за упите који чува у локалној бази.



Структура - јединствена база

- Захтев за изменом производа (1) уписује тренутно стање у локалну базу (2) и ажурира модел за упите (3) и затим објављује догађај (4).
- Сервис за упите чита ажуран модел за упите (5).



Предности

- Неминован код употребе обрасца *Event sourcing*.
- Боља подела надлежности.
- Једноставнији упитни модел. Боље перформансе упита. Подаци су најчешће денормализовани у циљу постизања оптималних перформанси.

Мане

- Увећана сложеност.
- Кашњење у репликацији. Коначна конзистентност (*Eventual Consistency*).
- Дуплирање података. Постоји могућност неконзистенције.

5.5 Комуникација

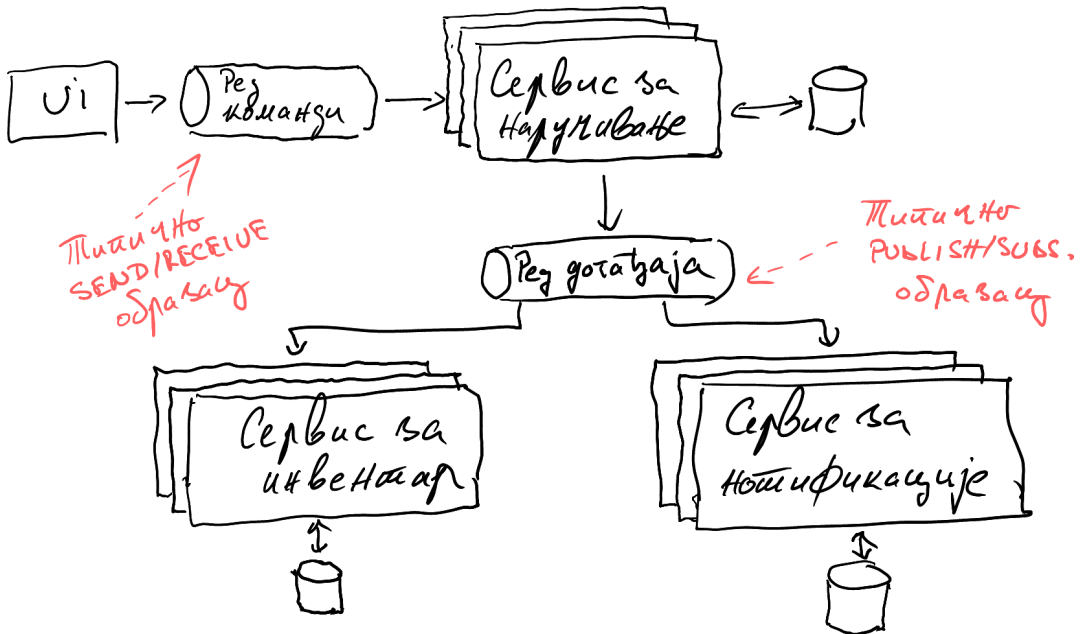
5.5.1 Messaging

- Слање порука је у основи архитектуре вођене догађајима.
- Асинхрона комуникација, слабо спрезање микросервиса.

- Посредник (*message broker*), који мора бити високо доступан, омогућава баферовање и перзистенцију порука.

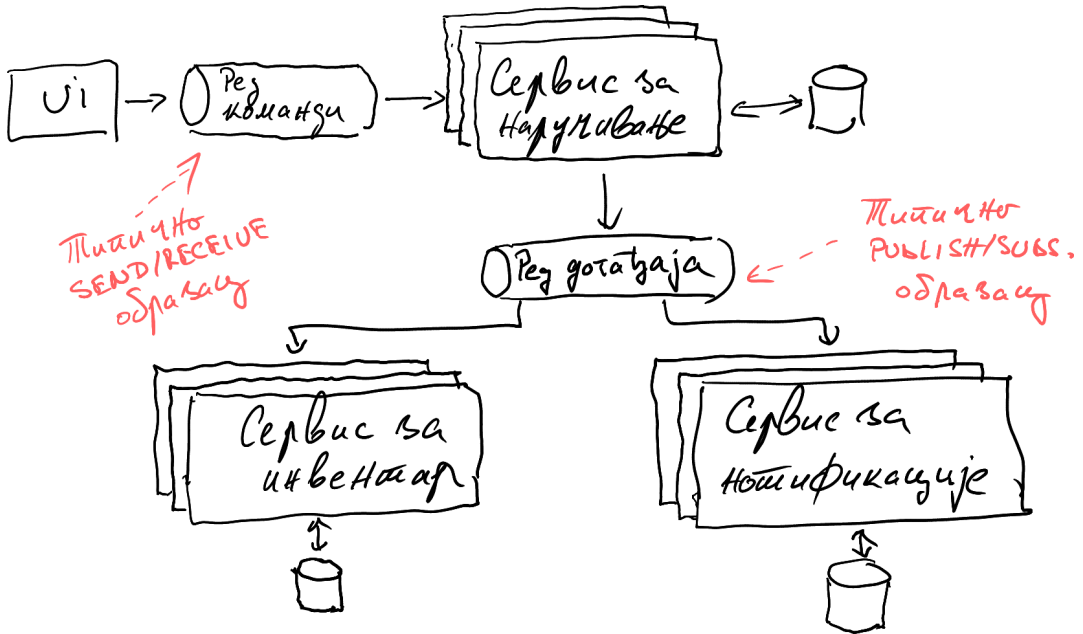
Send/Receive образац

- Обично представља комуникацију између два учесника (*point-to-point*) са специфичном наменом, најчешће извршење акције над циљним сервисом.
- Овај облик је коришћен типично од стране команди.
- Мора се обезбедити да само циљни сервис реагује на поруку.



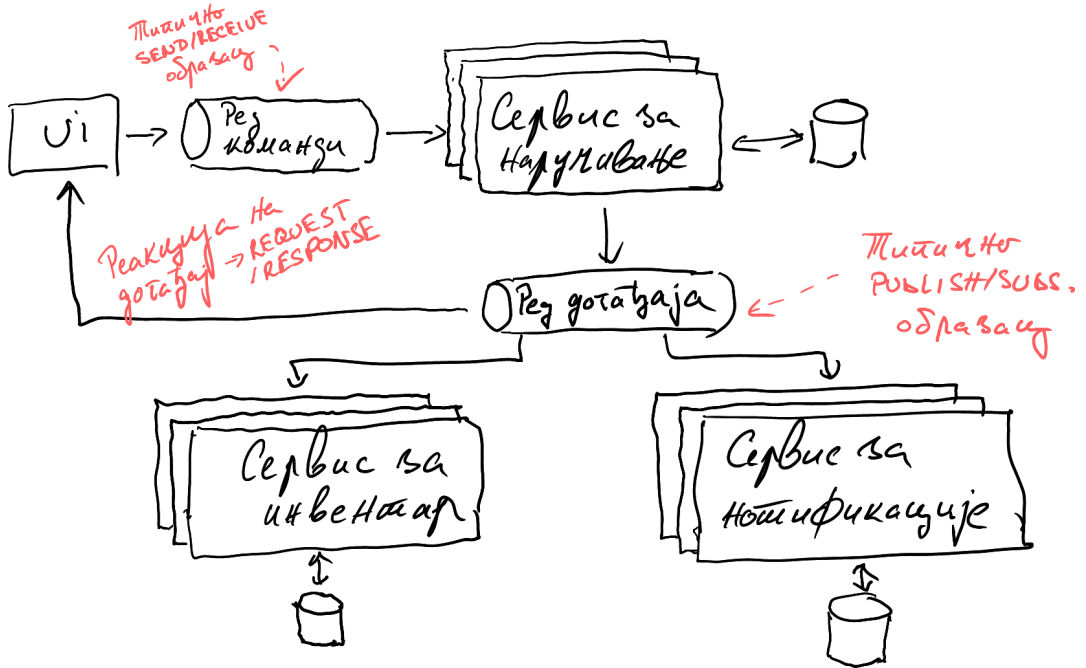
Publish/Subscribe образац

- Комуникација “један на више”.
- Сервиси заинтересовани за одређене поруке се региструју (*subscribe*).
- Сервиси примају поруке и обрађују их у паралели различитом брзином.
- Основа хоризонталног скалирања.



Request/Response образац

- Имплементира се када је потребна повратна информација, обично при имплементацији **Send/Receive** образаца.
- Порука/команда и одговор на команду могу бити корелирани одређеним идентификатором.



5.5.2 Remote Procedure Invocation (RPI)

- Сервиси често морају тесније сарађивати да би обрадили одређени захтев.
- Понекад је синхрони начин комуникације бољи. Тада користимо *RPI*.

Предности

- Једноставан вид комуникације. Синхрона варијанта *Request/Response* образац.

Мане

- Јако темпорално спрезање сервиса. Морају бити доступни истовремено.

Приступи

- **REST**⁵
 - Добро познат приступ. Основа комуникације на вебу.
 - Најчешће се користи у комбинацији са текстуалним порукама, нпр. JSON.
- **gRPC**⁶
 - Развијен у Гуглу.
 - Акценат на перформансама.

⁵https://en.wikipedia.org/wiki/Representational_state_transfer

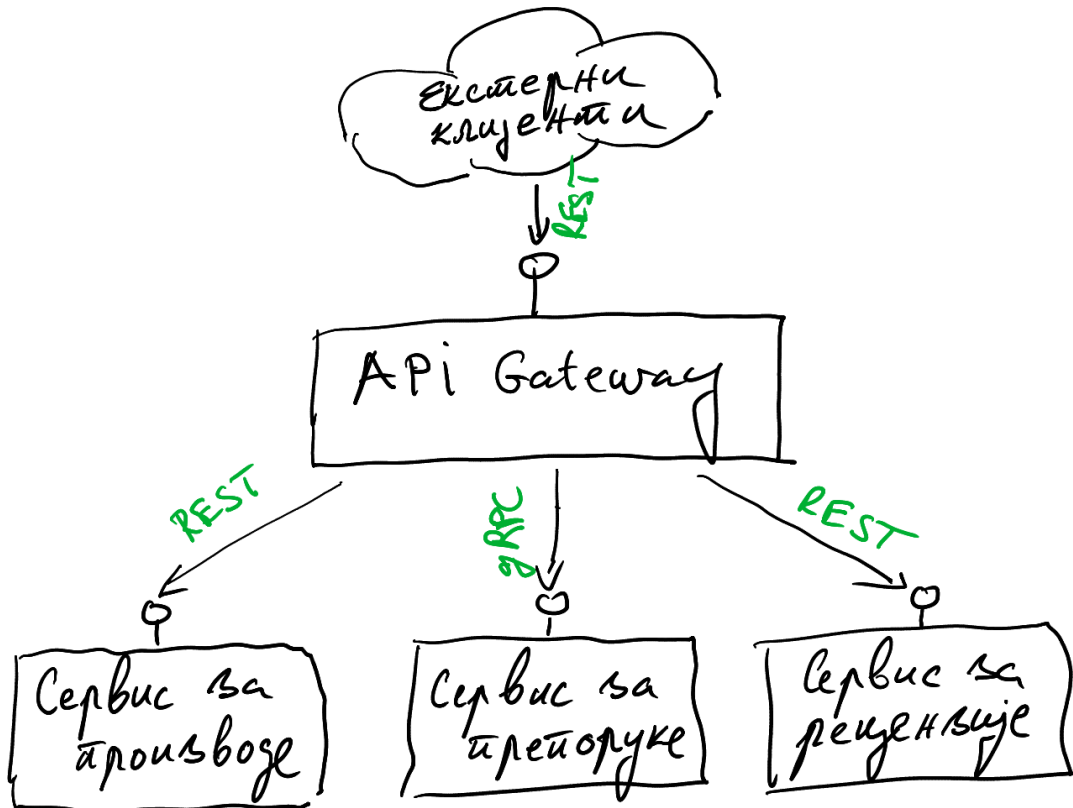
⁶<https://grpc.io/>

- Бинарне поруке базиране на технологији [Protocol Buffers](https://developers.google.com/protocol-buffers/)⁷.
- [Apache Thrift](https://thrift.apache.org/)⁸
 - Развијен у Фејсбуку.
 - Различити формати порука и транспортни протоколи.

5.5.3 API Gateway

- Инстанца класичног ОО обрасца *Facade*. Скривање интерне сложености.
- Посредник за спољне клијенте.
- Специјализација [обрасца API композиције](#) за екстерне клијенте. Интеграција података са више микросервиса.
- Може имплементирати додатне функционалности, нпр. ауторизацију.

Структура

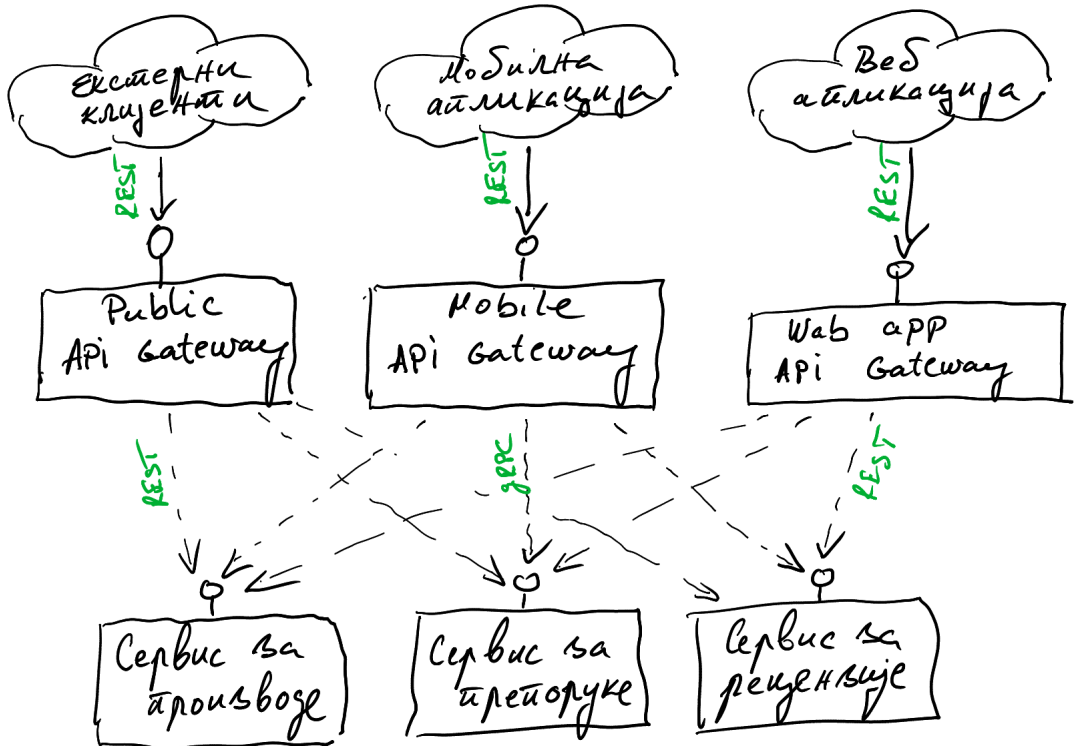


⁷<https://developers.google.com/protocol-buffers/>

⁸<https://thrift.apache.org/>

Варијанта - Backends for Frontends

- По један гејтвеј за сваки фронтенд.
- Специјализација API-ја.

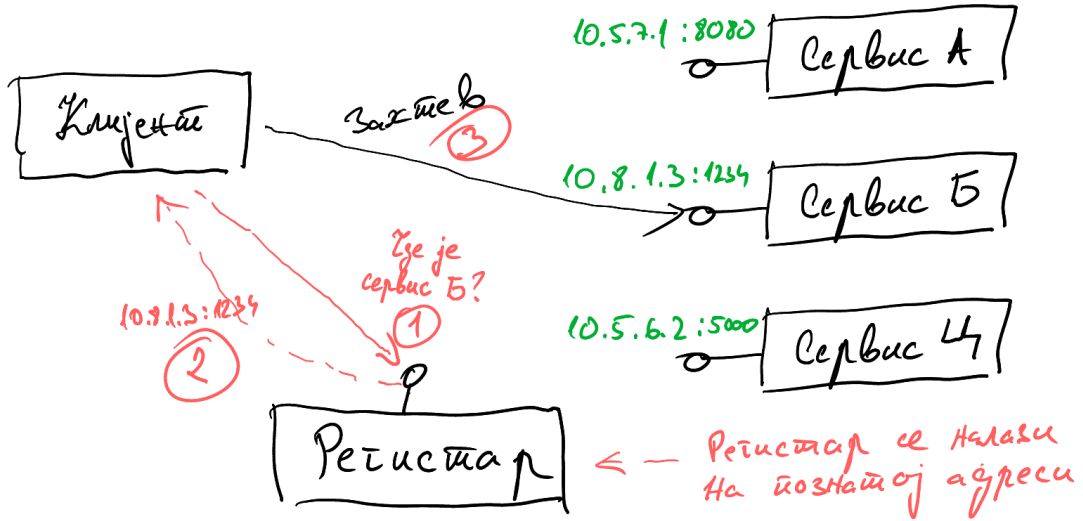
**Напомена**

- Једна тачка отказа. Обезбедити високу доступност.

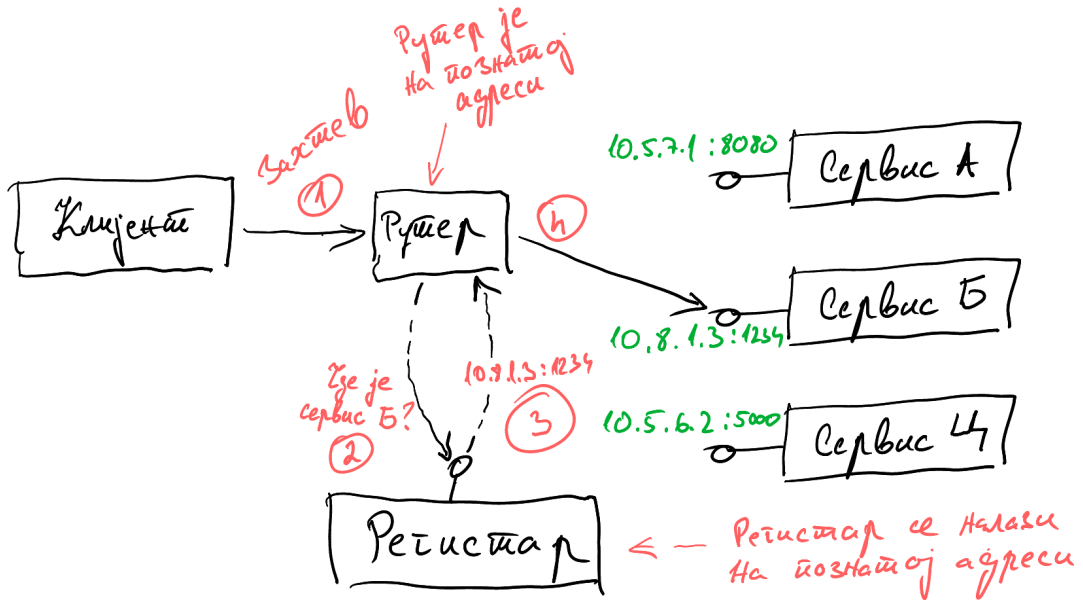
5.6 Откривање сервиса**5.6.1 Service Registry**

- За разлику од класичних дистрибуираних система код микросервисних архитектура сервиси нису увек на истој мрежној адреси.
- Како клијент сазнаје где се сервис налази?
- Специјални сервис који је увек на истој локацији и који има информације о локацијама свих других сервиса.
- Два приступа:
 - *клијентски* - код којег клијент сервис сам пита регистар,
 - *серверски* - код којег имамо посредника (*руишер*) који поставља питање регистру.

Структура - клијентски



Структура - серверски



5.7 Литература

- Hugo Filipe Oliveira Rocha, Practical Event-Driven Microservices Architecture, Apress, 2022.
- Microsoft, [Cloud Design Patterns](https://docs.microsoft.com/en-us/azure/architecture/patterns/)⁹

⁹<https://docs.microsoft.com/en-us/azure/architecture/patterns/>

- Chris Richardson, [Microservice Architecture](https://microservices.io/)¹⁰
- Wikipedia, [Microservices](https://en.wikipedia.org/wiki/Microservices)¹¹

¹⁰<https://microservices.io/>

¹¹<https://en.wikipedia.org/wiki/Microservices>

Глава 6

Развој базиран на компонентама

6.1 Razvoj softvera

In a world of rapidly changing business requirements, custom-made software is often too late -- too late to be productive before becoming obsolete.

- C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, Addison-Wesley Longman Publishing Co., Inc., 2002

6.2 Modularizacija

- Savladavanje kompleksnosti softvera podelom na manje celine -- *Divide and Conquer* princip.
- Moduli treba da imaju **visoku koheziju** i **slabu spregu** sa drugim modulima.

6.3 Softverske komponente

- Softverske komponente su izvršive jedinice koje se nezavisno proizvode, dobavljaju i instaliraju i koje se mogu komponovati u cilju formiranja funkcionalnog sistema.
- U cilju obezbeđivanja kompozicije, komponente moraju da poštuju određeni *komponentni model* i da budu kreirane za *određenu ciljnu platformu*.
- Sistem sastavljen od softverskih komponenti se naziva komponentni softver (*component software*).

6.4 Razvoj baziran na komponentama

- Razvoj baziran na komponentama (*Component-Based Development - CBD*) -- izrada softvera upotrebom softverskih komponenti.
- Komponentizacija nije nova ideja -- uspešno se primenjuje u različitim oblastima...
- ...ali do nedavno nije bilo velikog uspeha u oblasti razvoja softvera.

6.5 Šta je problem sa softverom?

Često se tvrdi da je softver previše fleksibilan da bi bilo moguće kreirati komponente -- ovo ne može biti razlog već više znak nezrelosti oblasti.

6.6 Integracija kao preduslov ponovne iskoristljivosti komponenti

A key issue is that today's software environments focus on writing new software, instead of integrating existing software into new systems. In reality, integrating existing code has become a large part of the work of software developers. Therefore, there is a need for tools that standardize the integration aspects of software so that reusing existing components becomes reliable, robust and cheap.

<http://www.osgi.org/About/Technology>

6.7 Preduslovi uspešne komponentizacije

- Postojanje standarda koji su praktični i ostvarljivi:
 - OSGi
 - EJB
 - DCOM i ActiveX
 - CORBA
- Postojanje tržišta komponenti:
 - Eclipse Marketplace
 - Firefox/Chrome priključci
 - Google Play, Apple App Store i dr.
- Isplativost razvoja: smatra se da komponenta mora bar 3 puta da se iskoristi da bi se njen razvoj isplatio.

6.8 Vrste softvera prema načinu dobavljanja

- Namenski izrađen softver (*Custom made*)
- Standardni softver

6.9 Namenski izrađen softver

- Može biti u potpunosti prilagođen korisniku.
- Može koristiti vlasnička znanja i iskustvo firme za koju se proizvodi.
- Ovim se može postići značajna prednost na tržištu.
- Razvoj i održavanje može biti skupo. Interoperabilnost sa drugim sistemima može biti problematično.

6.10 Standardni softver

- Kupuje se gotov i parametrizuje se da bude *dovoljno dobar* za posao za koji se koristi.
- Kratko vreme izgradnje -- svodi se na konfiguraciju i prilagođavanje.

- Unapređenje, održavanje i implementacija interoperabilnosti sa drugim sistemima se prebacuje na proizvođača standardnog softvera.
- Teško može biti osnova sticanja prednosti na tržištu jer i konkurencija može kupiti isti softver.

6.11 Problem sa standardnim softverom

- Često zahteva drastičnu promenu poslovnih procesa i načina rada u cilju prilagođavanja -- primer: uvođenje SAP R/3 u Australijsku poštu.
- Izmenu procesa bi trebalo raditi u skladu sa izmenom poslovnog okruženja a ne u cilju prilagođavanja predviđenom načinu rada softvera.

6.12 Komponentni softver -- hibridni pristup

- Namenski izraditi komponente u domenu ekspertize.
- Kupiti standardne komponente za ostale domene.
- Primer: kupovina GUI komponenti, komponenti za rad sa XML datotekama, bazama i sl.
- Dobijamo prednost na tržištu jer u svom domenu radimo posao na način na koji želimo ali istovremeno ubrzavamo razvoj upotrebom standardnih, robusnih komponenti u domenima koji nisu od presudnog značaja za naš poslovni domen.

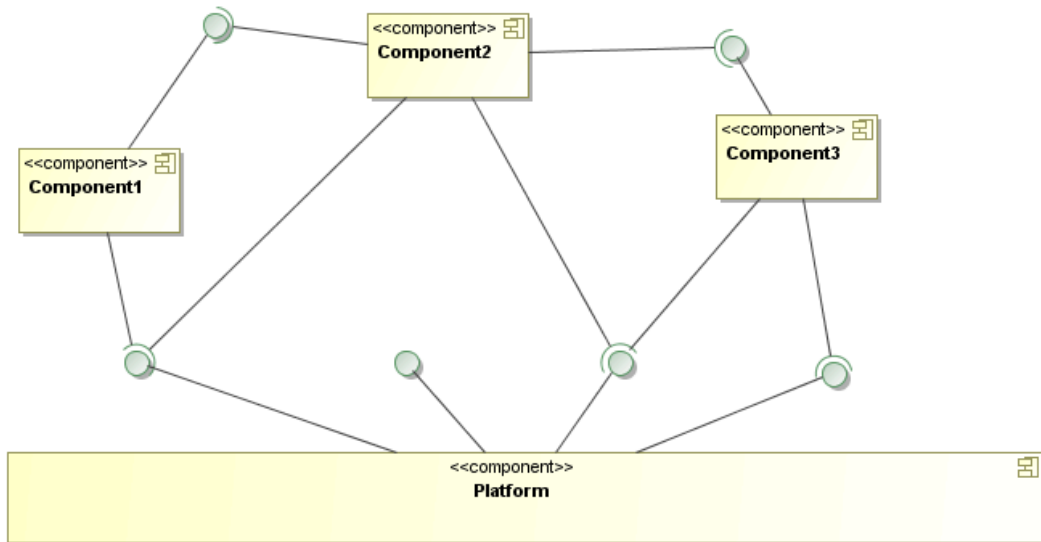
6.13 Evolutivni razvoj

- Za razliku od monolitnih rešenja gde imamo periodičnu zamenu sistema novim (revolutivni pristup) kod komponentnog razvoja unapređenje sistema možemo raditi evolutivno kroz nezavisno unapređenje komponenti.
- Imamo niz češćih manjih promena umesto redih velikih promena koje menjaju sistem iz korena.

6.14 Univerzalne komponente?

- ... ne postoje.
- Komponente se prave za određenu ciljnu platformu i moraju poštovati određeni komponentni model.
- Takođe, moramo unapred imati svest o korisnicima komponente koju razvijamo odnosno njihovim očekivanjima.

6.15 Apstraktna arhitektura softvera baziranog na komponentama



6.16 Platforma

- Obezbeđuje infrastrukturu u koju se *ugrađuju* komponente.
- Pruža skup bazičnih servisa koje komponente mogu da koriste.

6.17 Servisi

- Predstavlja skup funkcionalnosti koje određena komponenta pruža drugim komponentama i čije korišćenje je definisano ugovorom, koji propisuje način upotrebe (npr. interfejsi i protokoli), i ograničeno politikom upotrebe servisa.
- Platforma mora imati propisane mehanizme za objavljivanje i pronalaženje servisa.

6.18 Konzumenti i pružaoci servisa

- U komponentnom softveru svaka komponenta može biti u ulozi pružaoca i/ili konzumenta servisa.
- Različite tehnike pronalaženja i povezivanja servisa.

6.19 Literatura

- C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, Addison-Wesley Longman Publishing Co., Inc., 2002

Глава 7

Креирање компоненти у Пајтону

Sadržaj

7.1 Мотивација

- Пакет жели да омогући команду која се може позвати са терминала.
- Пакет жели да омогући покретање GUI апликације.
- Пакет жели да омогући прилагођавање своје функционалности употребом прикључака (*plugins*).

7.2 Конзолне скрипте

7.2.1 Организација пројекта

```
project_root_directory
├── pyproject.toml      # and/or setup.cfg, setup.py
├── src
│   └── timmins
│       ├── __init__.py
│       └── ...
```

У `__init__.py`:

```
def hello_world():
    print("Hello world")
```

7.2.2 Покретање као модул

У `src/timmins/__main__.py`:

```
from . import hello_world

if __name__ == '__main__':
    hello_world()
```

Сада је могуће покренути функцију са терминала на следећи начин:

```
$ python -m timmins
Hello world
```

7.2.3 Преко екстензионе тачке

У `pyproject.toml`:

```
[project.scripts]
hello-world = "timmins:hello_world"
```

Сада можемо скрипту директно позвати:

```
$ hello-world
Hello world
```

7.2.4 Исто тако можемо регистровати и GUI скрипту

```
[project.gui-scripts]
hello-world = "timmins:hello_world"
```

7.2.5 Генеричко решење

- Регистрација конзолних и GUI скрипти представља специјалну употребу општег механизма за проширење који називамо *Прикључне тачке*.

7.3 Прикључне тачке (*Entry Points*)

- Општи начин проширења функционалности пакета.
- Тип метаподатка који пакети могу да региструју.
- Други пакети могу динамички да открију и исчитају ове метаподатке.

7.3.1 Организација пројекта

Крећемо од исте организације пројекта.

```
project_root_directory
├─ pyproject.toml      # and/or setup.cfg, setup.py
├─ src
│   └─ timmins
│       ├── __init__.py
│       └─ ...
```

У `__init__.py`:

```
def hello_world():
    print("Hello world")
```

7.3.2 Поставка проблема

- Желимо да омогућимо прилагођен приказ основног стринга.
- Прилагођавање могу да ураде други пакети.

На пример испис може бити промењен на следећи начин:

```
!!! Hello world !!!
```

7.3.3 Први корак

Раздвајање приказа на две функције:

```
def display(text):
    print(text)

def hello_world():
    display('Hello world')
```

7.3.4 Откривање прикључака и модификоване display функције

```
from importlib.metadata import entry_points
display_eps = entry_points(group='timmins.display')
```

Прикључне тачке имају назив, групу и вредност (атрибути name, group, value):

```
(
    EntryPoint(name='excl', value='timmins_plugin_fancy:excl_display',
group='timmins.display'),
    ...,
)
```

7.3.5 Учитавање објекта регистрованог у прикључној тачки

```
display = display_eps[0].load()
```

Потребно је дефинисати понашање у случају да не постоји пакет који региструје функцију у прикључној тачки:

```
from importlib.metadata import entry_points
display_eps = entry_points(group='timmins.display')
try:
    display = display_eps[0].load()
except IndexError:
    def display(text):
        print(text)

def hello_world():
    display('Hello world')
```

7.3.6 Регистрација прилагођене display функције

```
timmins-plugin-fancy
├─ pyproject.toml      # and/or setup.cfg, setup.py
├─ src
│   └─ timmins_plugin_fancy
│       └─ __init__.py
```

У src/timmins_plugin_fancy/__init__.py:

```
def excl_display(text):
    print('!!!!', text, '!!!!')
```

pyproject.toml

```
# Note the quotes around timmins.display in order to escape the dot .
[project.entry-points."timmins.display"]
excl = "timmins_plugin_fancy:excl_display"
```

Сада можемо видети дејство прикључка уколико је пакет инсталиран:

```
>>> from timmins import hello_world

>>> hello_world()
!!! Hello world !!!
```

У случају да пакет `timmins_plugin_fancy` није инсталиран понашање је подразумевано:

```
>>> from timmins import hello_world

>>> hello_world()
Hello world
```

7.3.7 Регистрација више прикључака под истом групом

- Желимо да имамо више различитих начина исписа:

```
def excl_display(text):
    print('!!!', text, '!!!')

def lined_display(text):
    print(''.join(['-' for _ in text]))
    print(text)
    print(''.join(['-' for _ in text]))
```

`pyproject.toml`

```
[project.entry-points."timmins.display"]
excl = "timmins_plugin_fancy:excl_display"
lined = "timmins_plugin_fancy:lined_display"
```

Сада можемо у базном пакету урадити нешто попут овога:

```
display_eps = entry_points(group='timmins.display')
try:
    display = display_eps['lined'].load()
except KeyError:
    # if the 'lined' display is not available, use something else
    ...
```

Или можемо учитати све прикључке:

```
display_eps = entry_points(group='timmins.display')
for ep in display_eps:
    display = ep.load()
    # do something with display
...
```

7.4 Литература

- [Setuptools Entry Points](#)¹²
- [Creating and discovering plugins](#)¹³

¹²https://setuptools.pypa.io/en/latest/userguide/entry_point.html

¹³<https://packaging.python.org/en/latest/guides/creating-and-discovering-plugins/>